PENETRATION TESTING PROJECT

Penetration Testing Preparation and Reconnaissance Report

Environment Setup, Scope Validation and Passive Attack Surface Mapping

Item	Detail
Prepared by	Hrishikesh Agarwal
Report date	9 November 2026
Testing platform	Kali GNU/Linux Rolling 2026.3 (VirtualBox virtual machine)
Primary tools	Burp Suite Community 2026.10.1, OWASP ZAP 2.17.0, Firefox ESR 153.4.0, Wappalyzer, WhatWeb 0.6.4, curl 8.22.0, Shodan
Methodology	OWASP Web Security Testing Guide (WSTG)
Testing type	Passive reconnaissance only – no exploitation, brute force, fuzzing or active scanning
Document version	1.0
Classification	Confidential – Training Project

Table of Contents

No.	Section
1	Executive Summary
2	Scope, Authorization and Rules of Engagement
3	Methodology and Project Workflow
4	Task 1 – Environment Setup
5	Task 2 – Scope Validation and Test Planning
6	Task 3 – Reconnaissance and Attack Surface Mapping
7	Attack Surface Map
8	Summary of Observations
9	Limitations and Compliance Notes
10	Recommendations and Next Steps
11	Final Submission Checklist
A	Appendix A – Tool and System Versions
B	Appendix B – Project Folder Structure

Sections 4 to 6 follow the project order exactly: Task 1 first, Task 2 second and Task 3 third. Task 4 (final documentation) is this report.

1. Executive Summary

This report documents the preparation phase of an individual penetration testing project. It records, in order, how the testing environment was built, how the scope and test plan were prepared, and how passive reconnaissance was performed to understand technologies, HTTP communication and the visible attack surface of a website.

The work was completed on 9 November 2026 between approximately 18:33 and 20:07 (IST) inside a Kali Linux virtual machine. All activity was limited to passive observation and single, low-impact requests. No exploitation, brute force, fuzzing, directory discovery or active scanning was performed.

Key outcomes

- The environment was prepared: Kali Linux 2026.3, Firefox ESR 153.4.0, Burp Suite Community 2026.10.1, OWASP ZAP 2.17.0 and the Wappalyzer extension were installed and verified.
- Firefox was configured to send traffic through the Burp proxy at 127.0.0.1:8080, and the PortSwigger CA certificate was imported so HTTPS traffic can be inspected.
- A structured project folder (seven directories) and project documents were created: scope matrix, asset inventory, risk log and a WSTG-based test plan.
- Technology fingerprinting was performed with Wappalyzer, WhatWeb and curl; HTTP traffic was captured in Burp Suite; passive alerts were reviewed in OWASP ZAP.
- OWASP ZAP reported 24 passive alerts (0 high, 8 medium, 7 low, 9 informational) while browsing the practice site. These are passive observations and have not been validated as vulnerabilities.

> **Important:** The authorization fields in the scope documents are still marked **Pending**, and the reconnaissance in Task 3 was demonstrated on public websites rather than on a formally assigned application. This report therefore presents a **preparation and tool-validation exercise**. Section 9 explains what must be completed before any real engagement.

2. Scope, Authorization and Rules of Engagement

A penetration test must only be performed against systems for which written permission exists. The table below shows the authorization record as it stands in the project files.

Information	Required	Current status
Assigned URL	Assigned application URL (example: https://example.com)	Placeholder values in project files; to be replaced with the real URL
Approved domain	Domain confirmed by the project coordinator	Pending confirmation
Testing environment	Staging or approved production	Not yet confirmed
Testing account	Authorized test account	Not yet issued
Testing period	Approved dates and times	Not yet confirmed
Allowed activities	Manual browsing and passive analysis	Proposed in the scope matrix
Excluded activities	DoS, brute force, exploitation, fuzzing	Proposed in the scope matrix
Emergency contact	Project coordinator	To be recorded
Rules of engagement approval	Written approval	Pending

Rules applied throughout this project

- No active scans, brute force, exploitation, fuzzing or intrusive testing.
- Keep Burp Suite Intercept switched off during normal browsing.
- Do not guess hidden URLs or perform directory discovery.
- Do not send state-changing requests or replay modified requests without approval.
- Redact passwords, tokens, personal data and session cookies from all evidence.
- Use ZAP for passive analysis only; do not use Attack or Active Scan.

3. Methodology and Project Workflow

The project follows the OWASP Web Security Testing Guide (WSTG) as its methodology. The work is divided into three technical tasks followed by documentation.

Order	Activity	Tool
1	Update Linux	Terminal
2	Install required tools	APT / pipx
3	Configure Burp Suite	Burp Suite
4	Configure browser proxy	Firefox
5	Install CA certificate	Firefox
6	Create documentation folders	Terminal
7	Confirm authorization and scope	Project documents
8	Prepare scope matrix and asset inventory	CSV files
9	Prepare WSTG test plan and risk log	WSTG / CSV files
10	Verify URL and redirects	curl
11	Identify technologies	Wappalyzer / WhatWeb
12	Browse application manually	Firefox
13	Capture HTTP traffic	Burp Suite
14	Identify endpoints and functionality	Burp / Firefox
15	Review passive findings	OWASP ZAP / Shodan
16	Prepare attack surface map	Diagram
17	Prepare final report	This document

4. Task 1 – Environment Setup

The objective of Task 1 is to prepare Linux, install the required tools, configure Burp Suite and Firefox, and confirm that HTTPS traffic can be inspected.

Step 1. Update and verify the Linux system

The Kali Linux virtual machine (named *kali 123* in Oracle VirtualBox) was opened and the system was updated and checked with the following commands.

```
sudo apt update
sudo apt upgrade -y
cat /etc/os-release
df -h
free -h
```

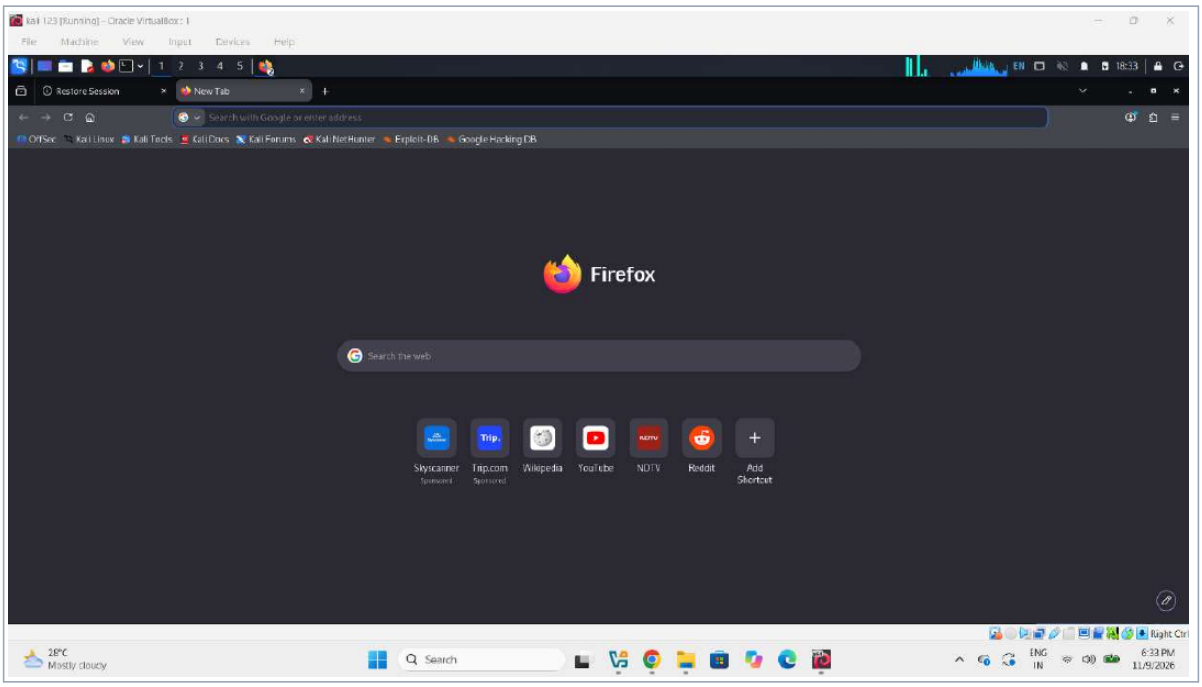


Figure 1: Kali Linux virtual machine running in VirtualBox with Firefox open.

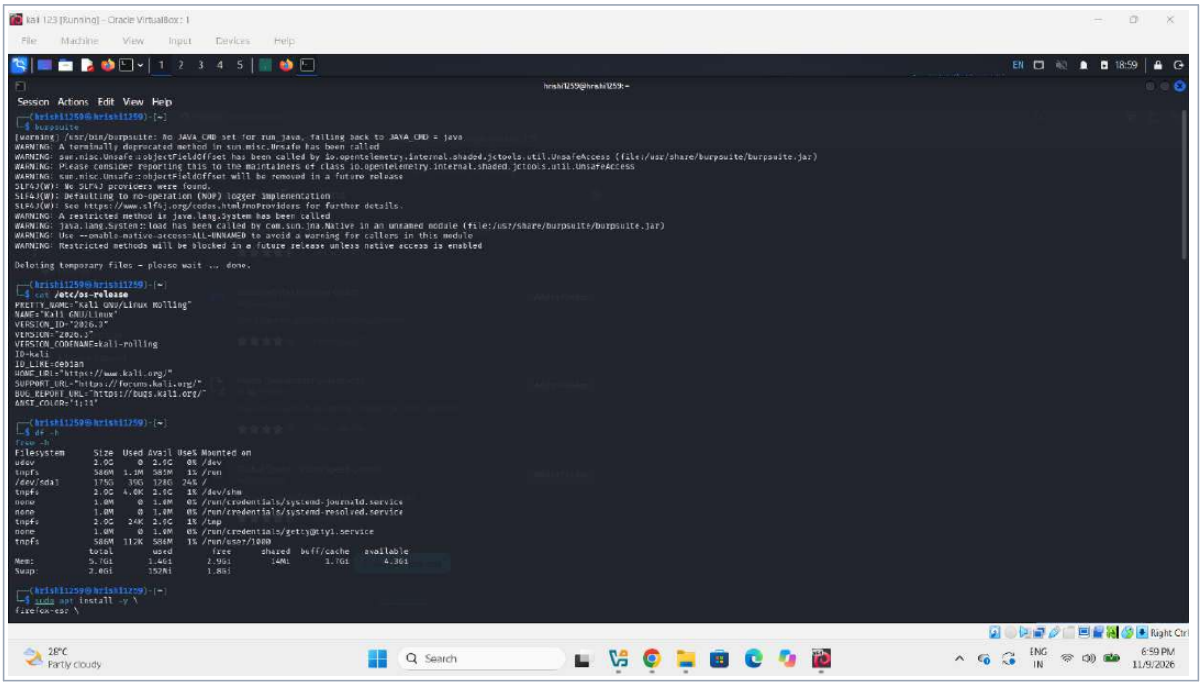


Figure 2: Terminal output showing /etc/os-release (Kali GNU/Linux Rolling 2026.3), disk space and memory.

Check	Result
Operating system	Kali GNU/Linux Rolling, version 2026.3
Root disk	175 GB total, 39 GB used, 128 GB available (24 percent used)
Memory	5.7 GiB total, 4.3 GiB available
Swap	2.0 GiB

Step 2. Install the required tools

The basic tools were installed with APT: Firefox ESR, curl, wget, git, Python 3, pip, venv, pipx, WhatWeb, jq, unzip, nano and tree. Most were already at the newest version. The installations were then verified.

```
sudo apt install -y firefox-esr curl wget git python3 python3-pip \
  python3-venv pipx whatweb jq unzip nano tree

firefox --version
curl --version
whatweb --version
python3 --version
```

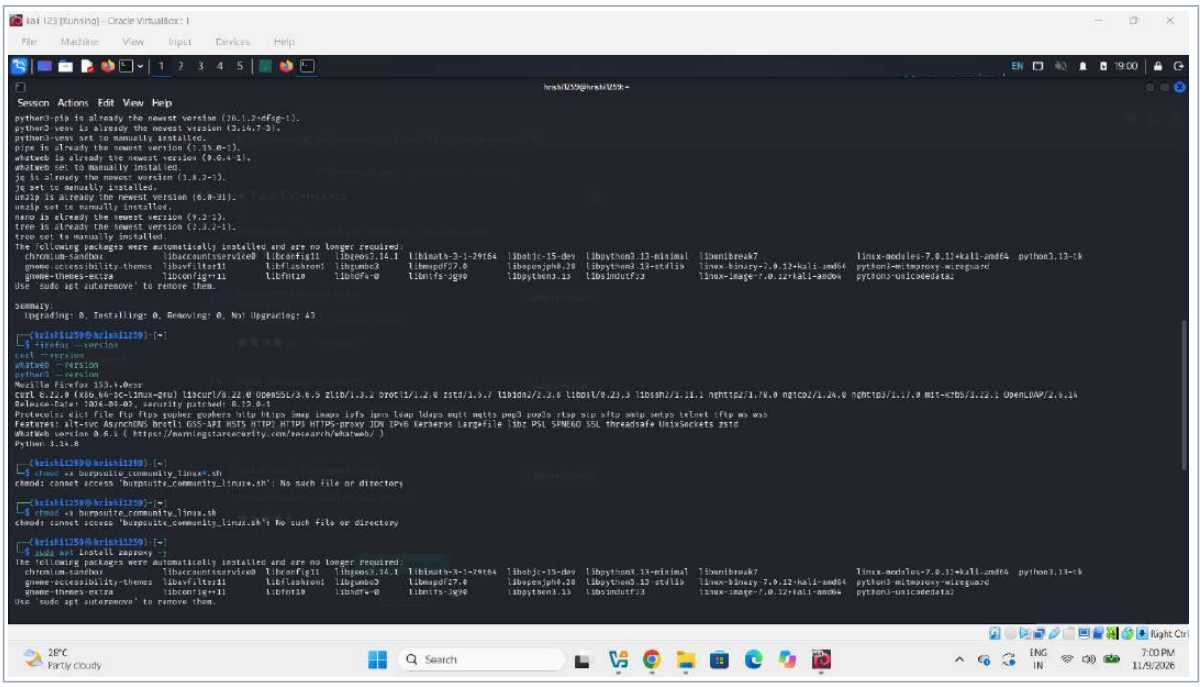


Figure 3: Verification of installed versions: Firefox 153.4.0esr, curl 8.22.0, WhatWeb 0.6.4 and Python 3.14.8.

Step 3. Install Burp Suite Community Edition

Burp Suite is used to intercept and inspect HTTP and HTTPS requests and responses. The standalone installer script (`burpsuite_linux_v2026_10_1.sh`) was downloaded, but the `chmod` command on a generic file name returned *No such file or directory* because the actual file name differs. Burp Suite was then installed from the Kali repository instead and launched with the `burpsuite` command.

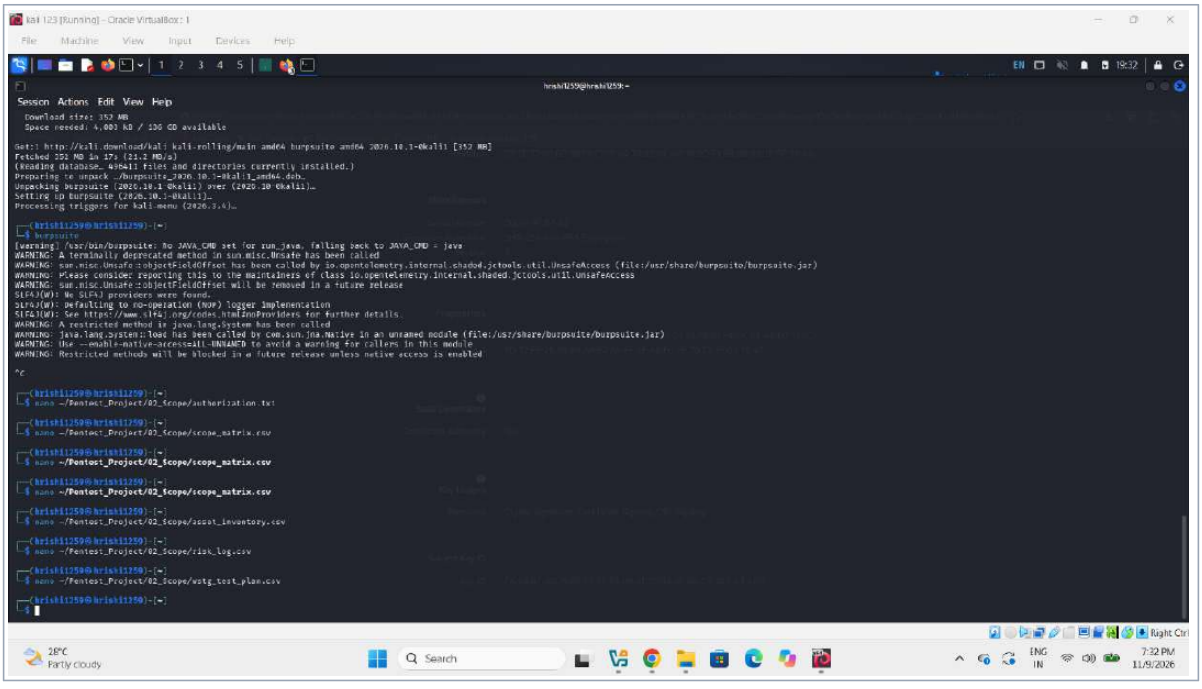


Figure 4: APT installation of Burp Suite 2026.10.1 and the project file commands in the terminal.

When Burp Suite started, a temporary project with Burp defaults was selected. The proxy listener settings are shown below.

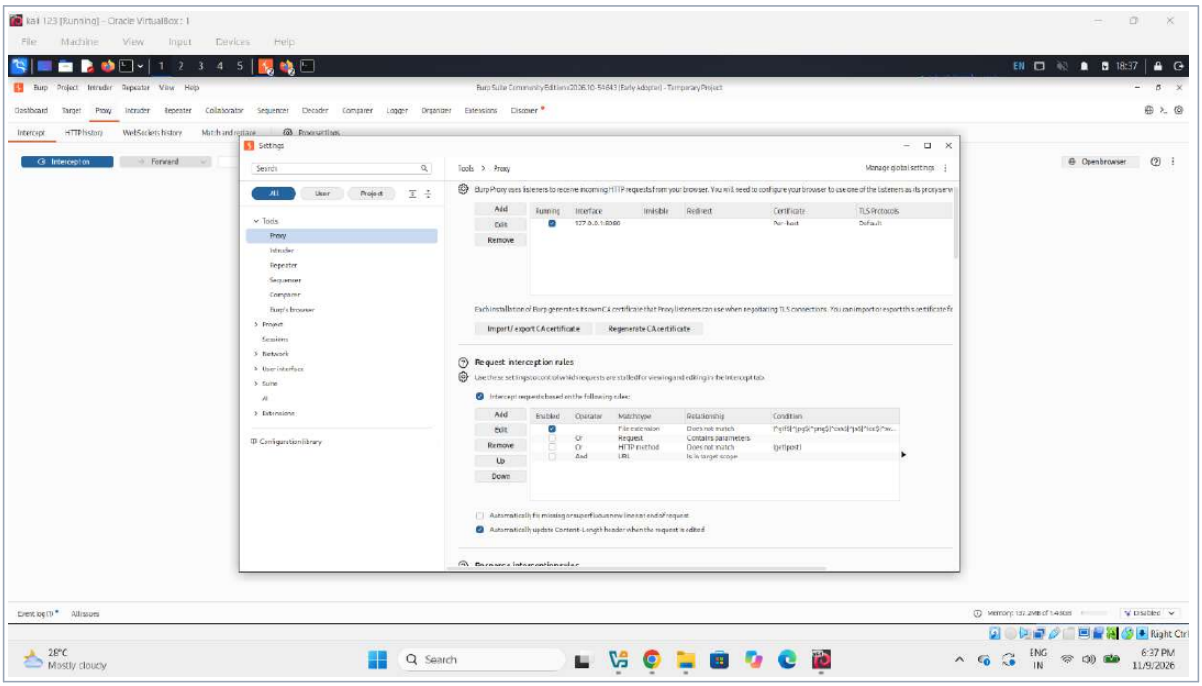


Figure 5: Burp Suite Settings: the proxy listener is running on 127.0.0.1:8080, with request interception rules visible.

Step 4. Install OWASP ZAP

OWASP ZAP was installed with `sudo apt install zaproxy -y`. The official download page (ZAP 2.17.0) was also opened for reference. ZAP is used for passive analysis only.

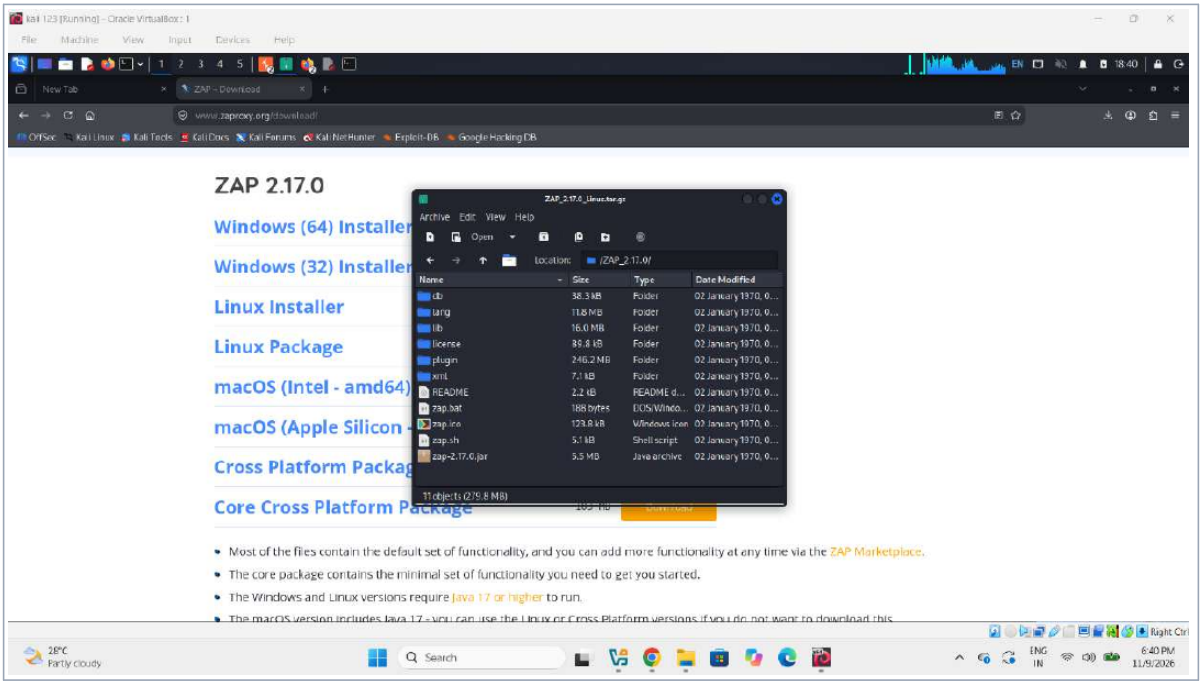


Figure 6: Official OWASP ZAP 2.17.0 download page, with the Linux package contents shown.

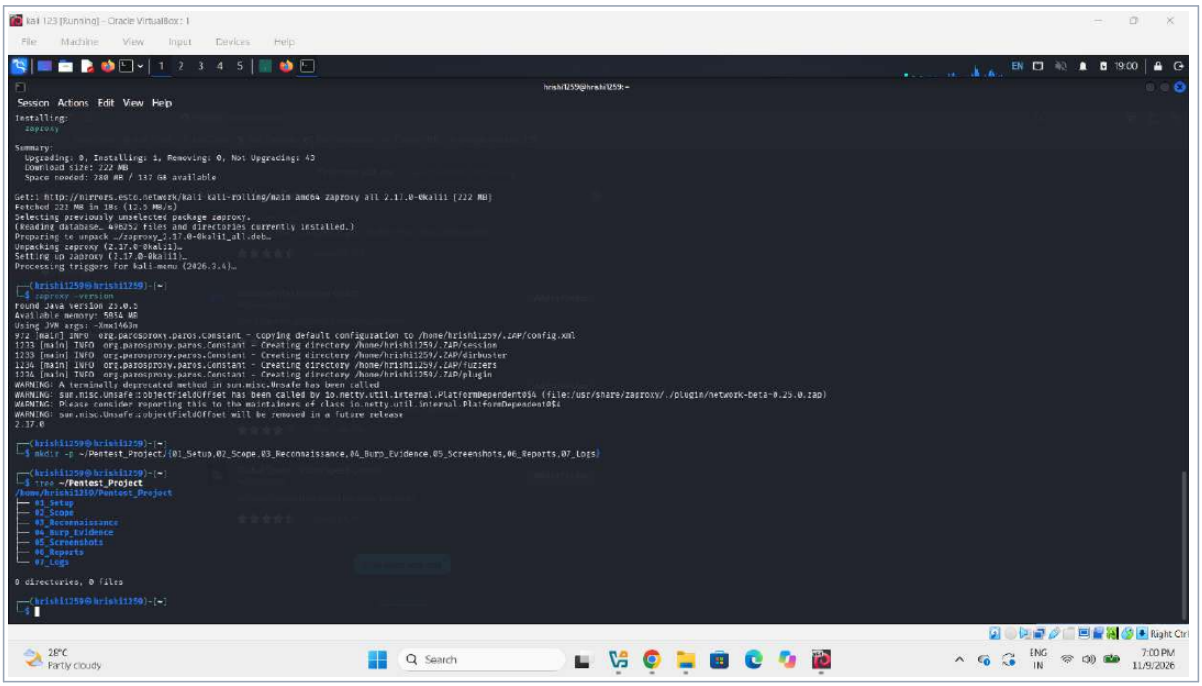


Figure 7: ZAP 2.17.0 installed (zaproxy -version) and the project folder structure created with mkdir and tree.

Step 5. Install Wappalyzer in Firefox

The Wappalyzer extension identifies web servers, frameworks, JavaScript libraries, analytics tools and other technologies. After installation, the extension displayed its data-collection notice and requested additional permissions.

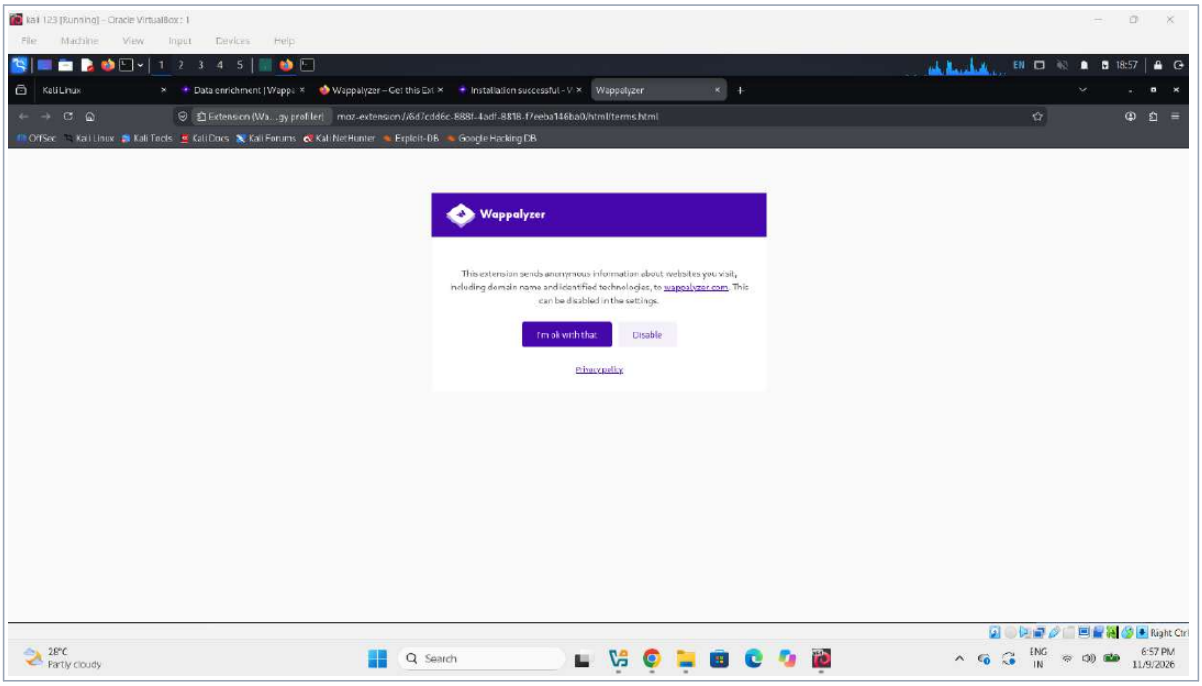


Figure 8: Wappalyzer terms page: the extension sends anonymous site information to wappalyzer.com.

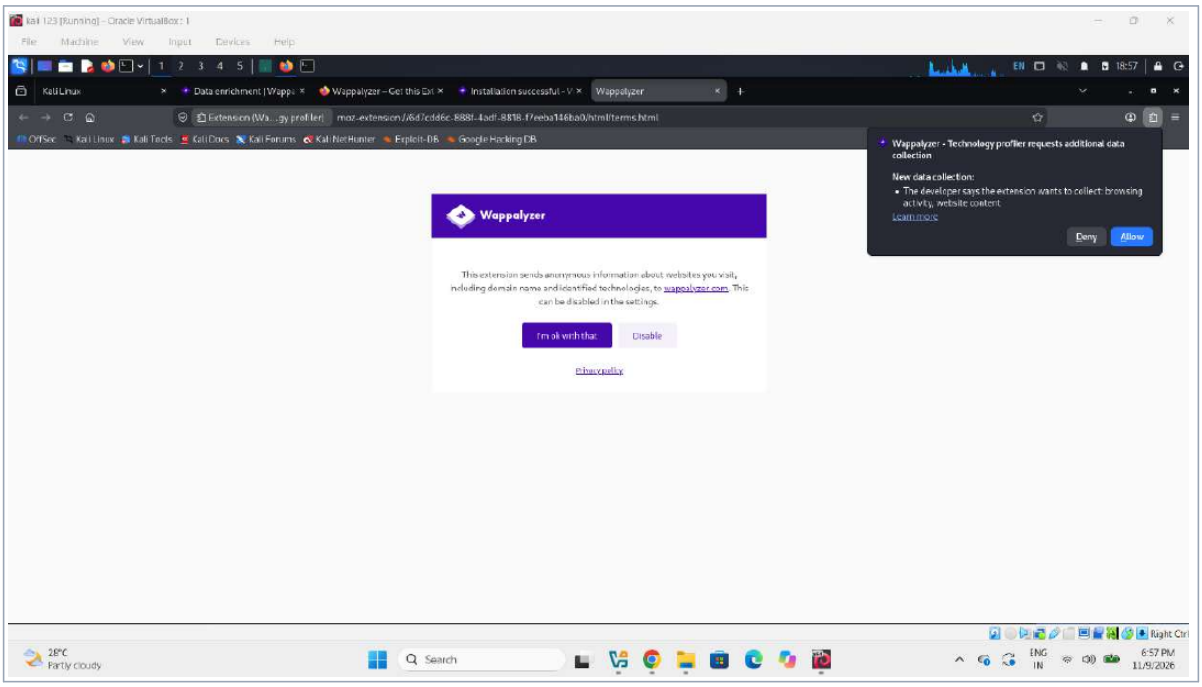


Figure 9: Firefox permission prompt: Wappalyzer requests browsing activity and website content (Deny or Allow).

Privacy note: Wappalyzer sends the domain names and detected technologies of the sites you visit to a third-party service. For real client engagements, check whether this is acceptable under the rules of engagement, or disable data sharing in the extension settings.

Step 6. Create the project folder and notes

```
mkdir -p ~/Pentest_Project/{01_Setup,02_Scope,03_Reconnaissance,04_Burp_Evidence,05_Screenshots,06_Repor
tree ~/Pentest_Project
```

The tree command confirmed seven directories and no files. A project notes file was then created with the following headings: Project Name, Assigned Application, Assigned URL, Operating System, Browser, Burp Suite Version, OWASP ZAP Version, Project Start Date, Testing Authorization and Rules of Engagement.

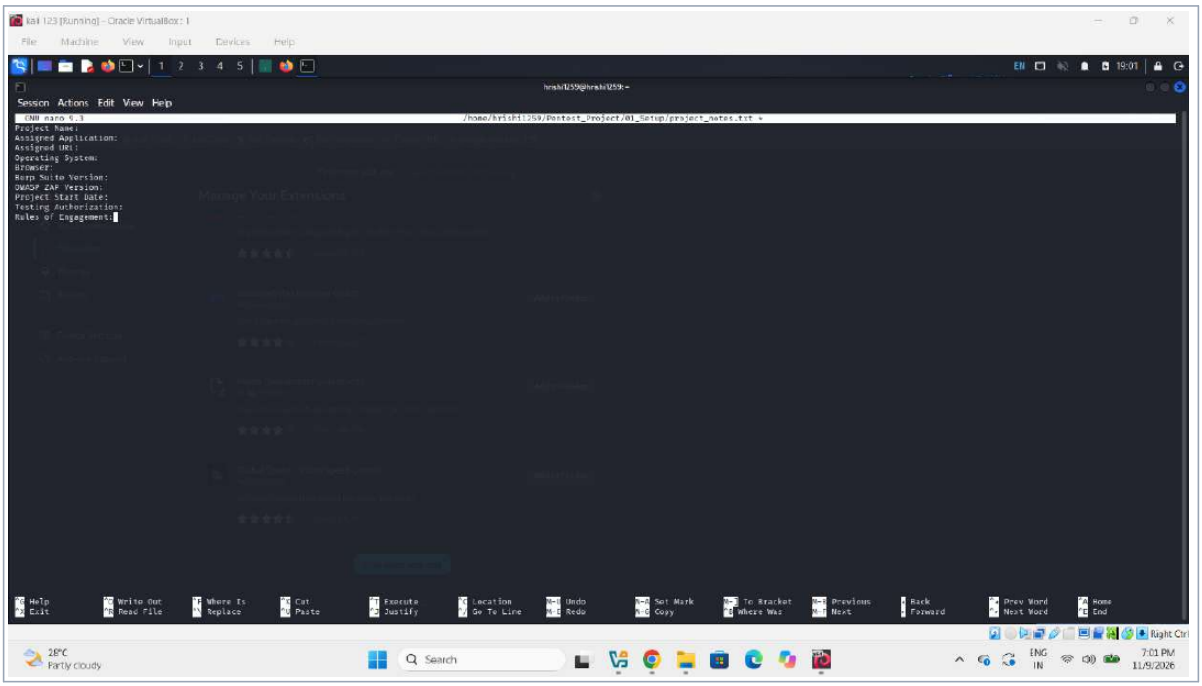


Figure 10: Project notes template in nano (01_Setup/project_notes.txt).

Step 7. Configure Firefox to use Burp Suite

In Firefox, **Settings > Network Settings > Manual proxy configuration** was used to point the browser at the Burp listener.

Setting	Value
HTTP Proxy	127.0.0.1
Port	8080
Also use this proxy for HTTPS	Enabled
HTTPS Proxy	127.0.0.1 (port 8080)

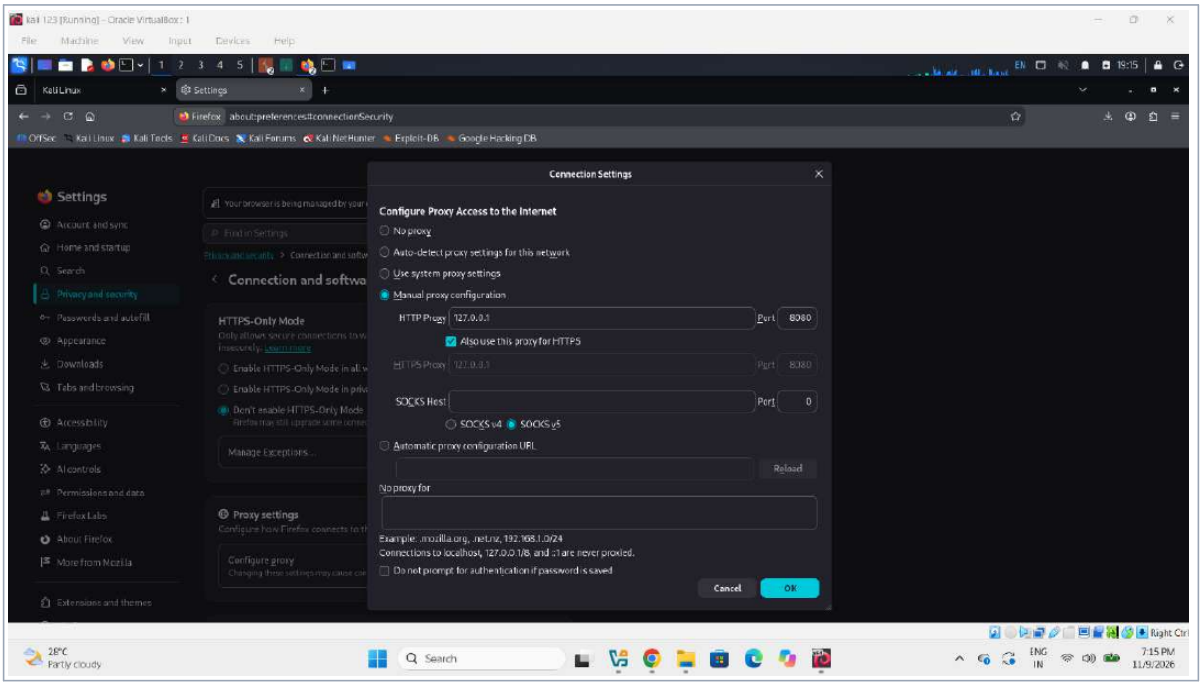


Figure 11: Firefox Connection Settings with manual proxy configuration set to 127.0.0.1:8080.

Recommendation: Use a dedicated Firefox testing profile rather than an everyday browsing profile, so that proxy settings and the Burp certificate do not affect normal browsing.

Step 8. Install the Burp CA certificate in Firefox

This step allows HTTPS traffic to be inspected without certificate warnings. With Burp running and Firefox proxied, the page `http://burp` was opened and the CA certificate (`cacert.der`, 987 bytes) was downloaded.

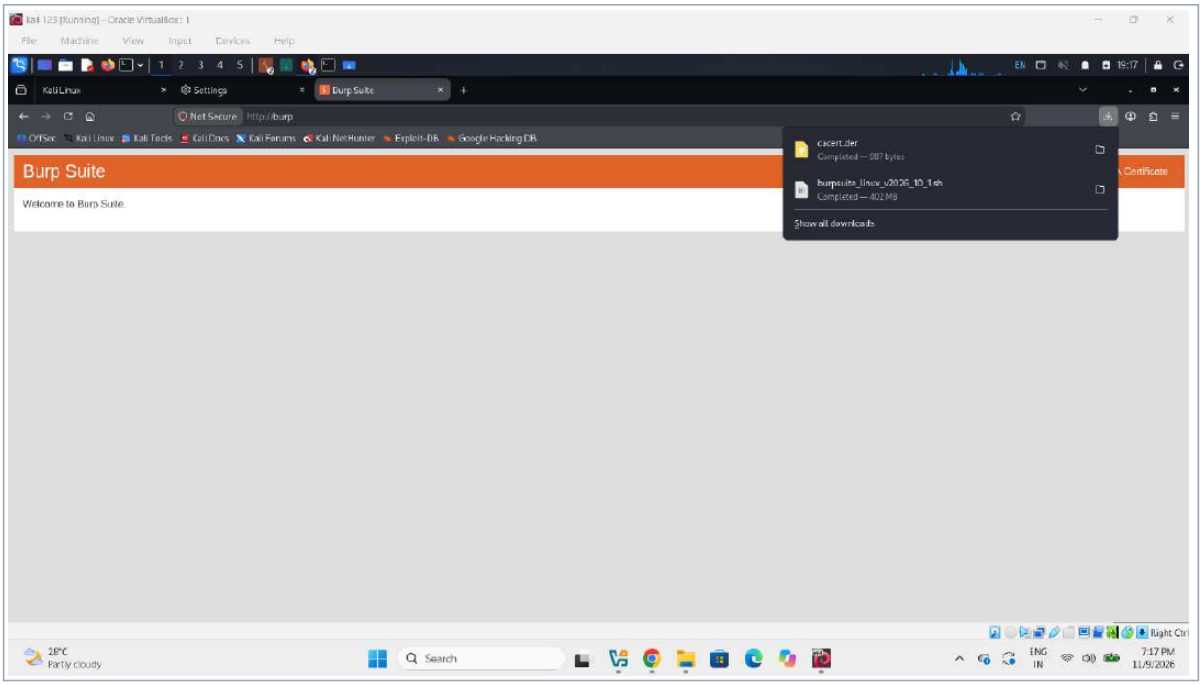


Figure 12: Burp Suite welcome page at http://burp with cacert.der downloaded.

The certificate was then imported through **Settings > Privacy and Security > Certificates > View Certificates > Authorities > Import**.

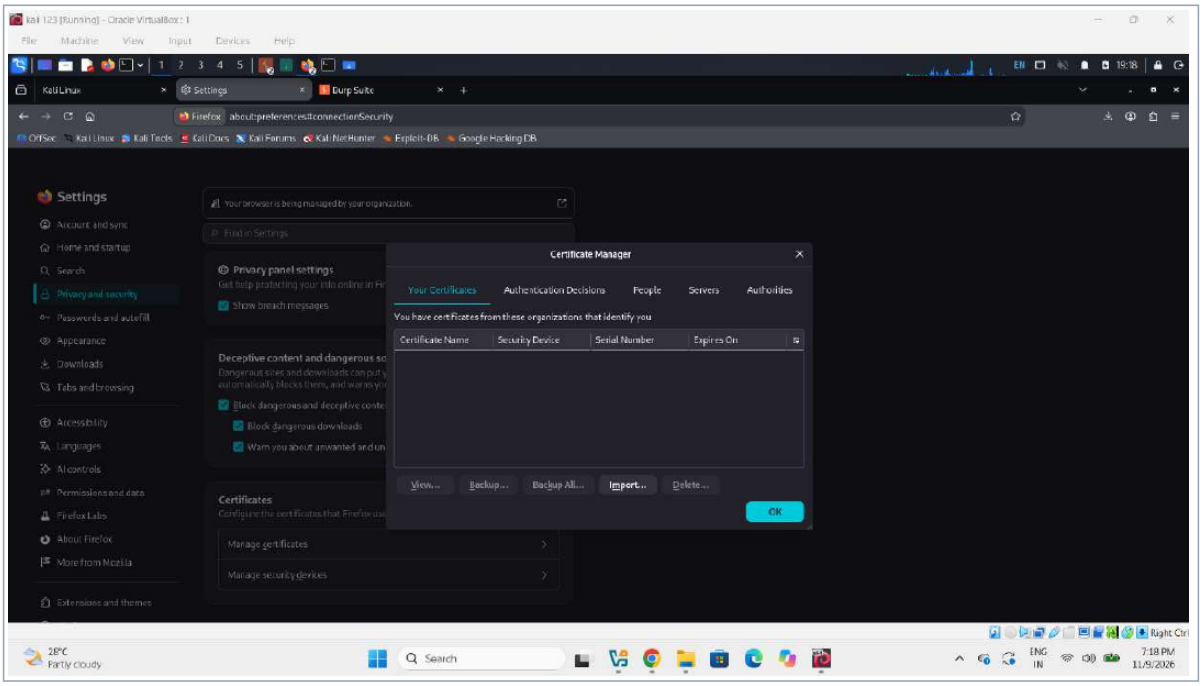


Figure 13: Firefox Certificate Manager before import.

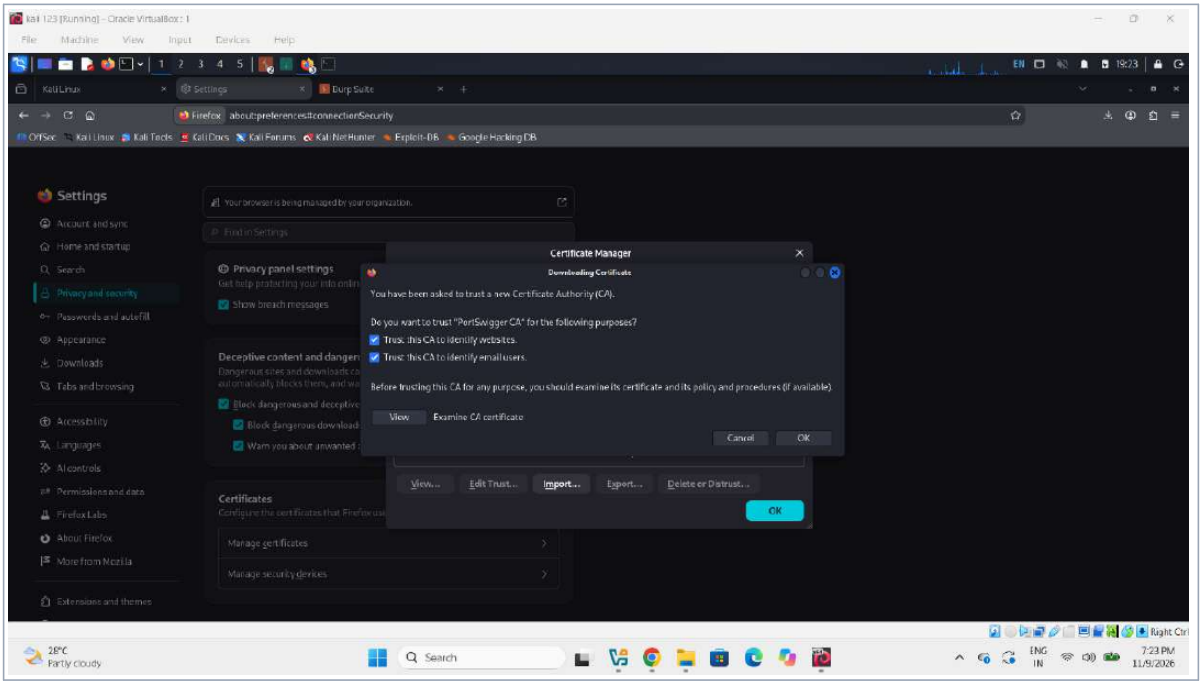


Figure 14: Import dialog: Firefox asks whether to trust PortSwigger CA to identify websites and email users.

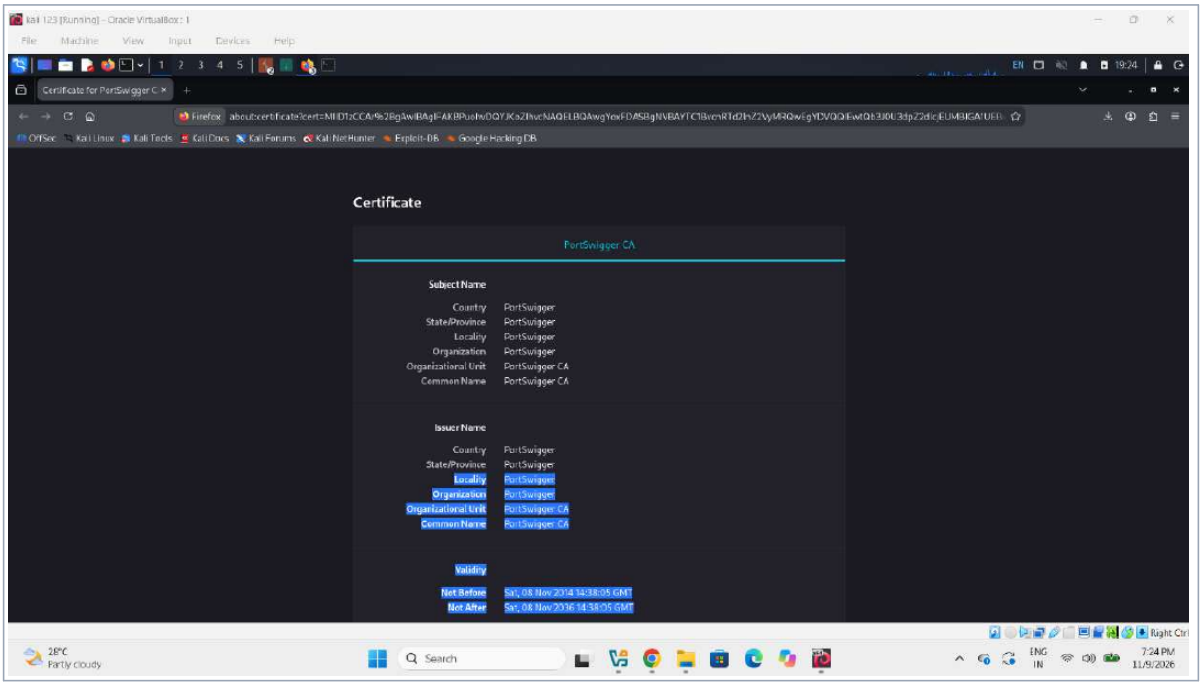


Figure 15: Certificate details (1): subject and issuer are PortSwigger CA; valid from 8 Nov 2014 to 8 Nov 2036.

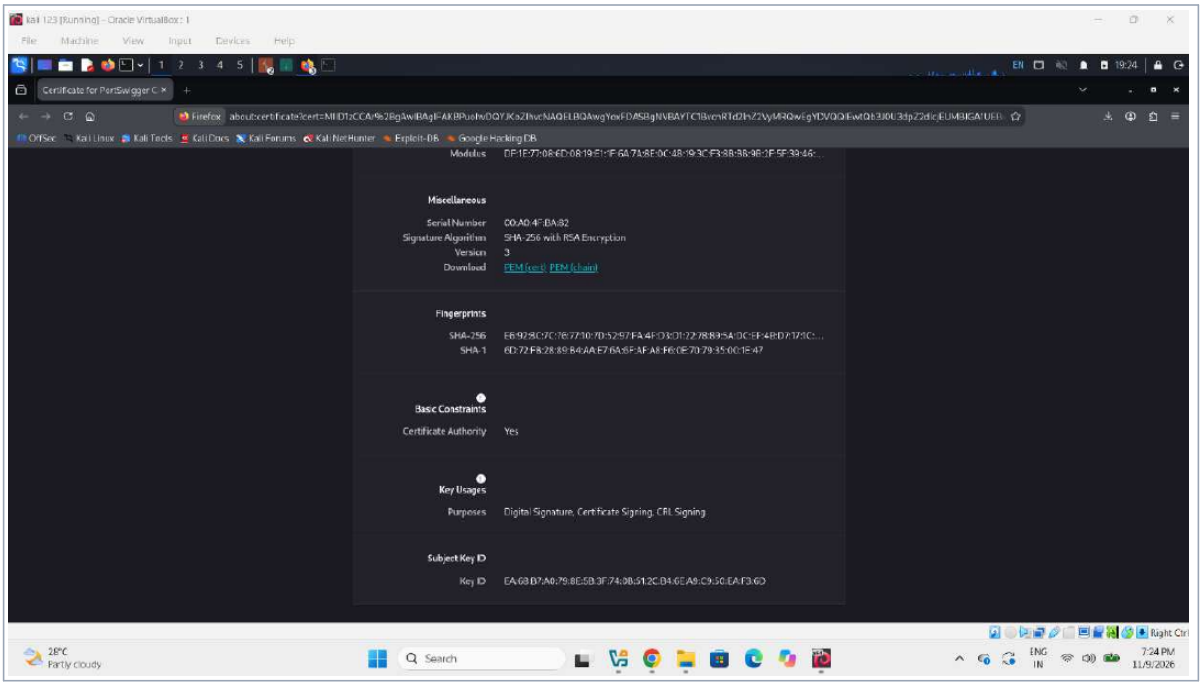


Figure 16: Certificate details (2): SHA-256 with RSA, Certificate Authority = Yes, key usages Digital Signature, Certificate Signing and CRL Signing.

Verification. After import, Burp **Proxy > HTTP history** showed HTTPS requests (the TLS column is ticked) without certificate errors, which confirms that HTTPS inspection works (see Figure 31).

> **Security note:** Trust the Burp CA only in the dedicated testing profile and for identifying websites. Trusting it for email users is not needed. Remove the certificate when the project is finished.

5. Task 2 – Scope Validation and Test Planning

Task 2 started only after Task 1 was complete. The objective is to define exactly what may be tested before any testing begins. Having an assigned URL is not, by itself, authorization to test.

Step 9. Collect application authorization details

An authorization file was created at `02_Scope/authorization.txt` to record the approved URL, domain, environment, test account, testing period, allowed and excluded activities, emergency contact and written approval (see Section 2 for the current status).

Step 10. Create the scope matrix

The scope matrix lists each asset, the approved action, the excluded action, the owner approval status and notes.

Asset	Approved action	Excluded action	Owner approval	Notes
Main Website	Manual browsing	DoS testing	Pending	Confirm scope
Login Page	View login functionality	Brute force	Pending	Use authorized account
Search Function	Manual observation	Automated fuzzing	Pending	Confirm limitations
API Endpoints	Passive observation	Unauthorized API calls	Pending	Confirm API scope
Admin Panel	Only if explicitly authorized	Privilege escalation	Pending	Restricted access

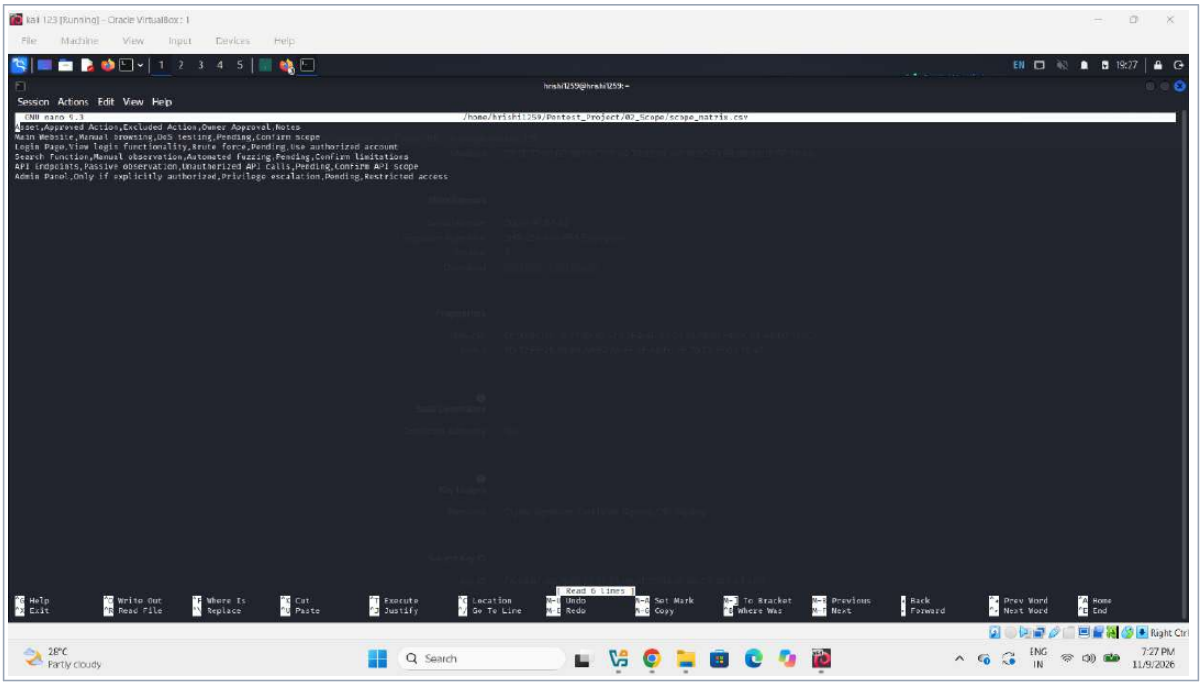


Figure 17: scope_matrix.csv opened in nano.

Step 11. Prepare the asset inventory

The asset inventory lists approved application components. The entries below use the placeholder domain example.com and must be replaced with the real approved assets.

Asset ID	Asset name	URL	Type	Owner	Scope status
A01	Main Website	https://example.com	Web Application	Web Team	Pending
A02	Login Page	https://example.com/login	Web Page	Web Team	Pending
A03	Search Function	https://example.com/search	Functionality	Web Team	Pending
A04	API	https://api.example.com	API	API Team	Pending

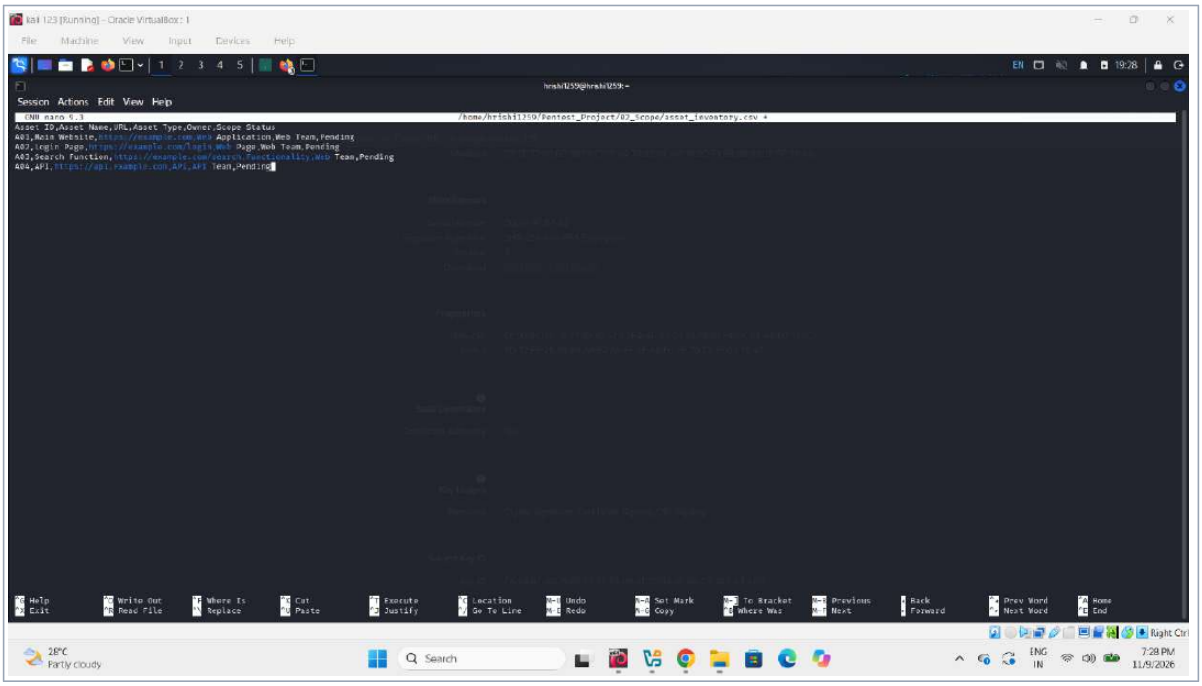


Figure 18: asset_inventory.csv opened in nano.

Step 12. Prepare the dependency and risk log

The risk log records how testing could affect availability or data, and how each risk is mitigated. A file named `02_Scope/risk_log.csv` was created for this purpose (see the command history in Figure 21).

Risk ID	Risk	Impact	Mitigation	Approval required
R01	Application downtime	High	No availability testing	Yes
R02	Account lockout	Medium	No repeated login attempts	Yes
R03	Third-party service impact	High	Exclude third-party assets	Yes
R04	Session disruption	Medium	Use approved test accounts	Yes
R05	Sensitive data exposure	High	Do not download unnecessary data	Yes

Step 13. Prepare the OWASP WSTG test plan

The WSTG categories below are mapped into the plan. Mapping a category does not authorize the corresponding test; each activity still needs approval.

WSTG category	What will be assessed later
Information Gathering	Application architecture and exposed information
Configuration and Deployment Management	Security configurations
Identity Management	User identity and registration
Authentication	Login and authentication mechanisms
Authorization	Access control
Session Management	Cookies and sessions
Input Validation	Input handling
Error Handling	Error messages and behavior
Cryptography	HTTPS and cryptographic protections
Business Logic	Application workflow
Client-side Testing	Browser-side functionality

The initial test plan (`wstg_test_plan.csv`) contains only passive or observation-based activities:

WSTG category	Assessment objective	Target	Method	Status
Information Gathering	Identify application technologies	Approved website	Passive	Planned
Configuration Management	Observe security headers	Approved website	Passive	Planned
Authentication	Document login functionality	Login page	Manual observation	Planned
Authorization	Identify role-based functions	Approved account	Manual observation	Planned
Session Management	Document cookie attributes	Approved website	Passive	Planned
Input Validation	Identify input fields	Forms	Inventory only	Planned
Error Handling	Document visible errors	Approved website	Observation only	Planned

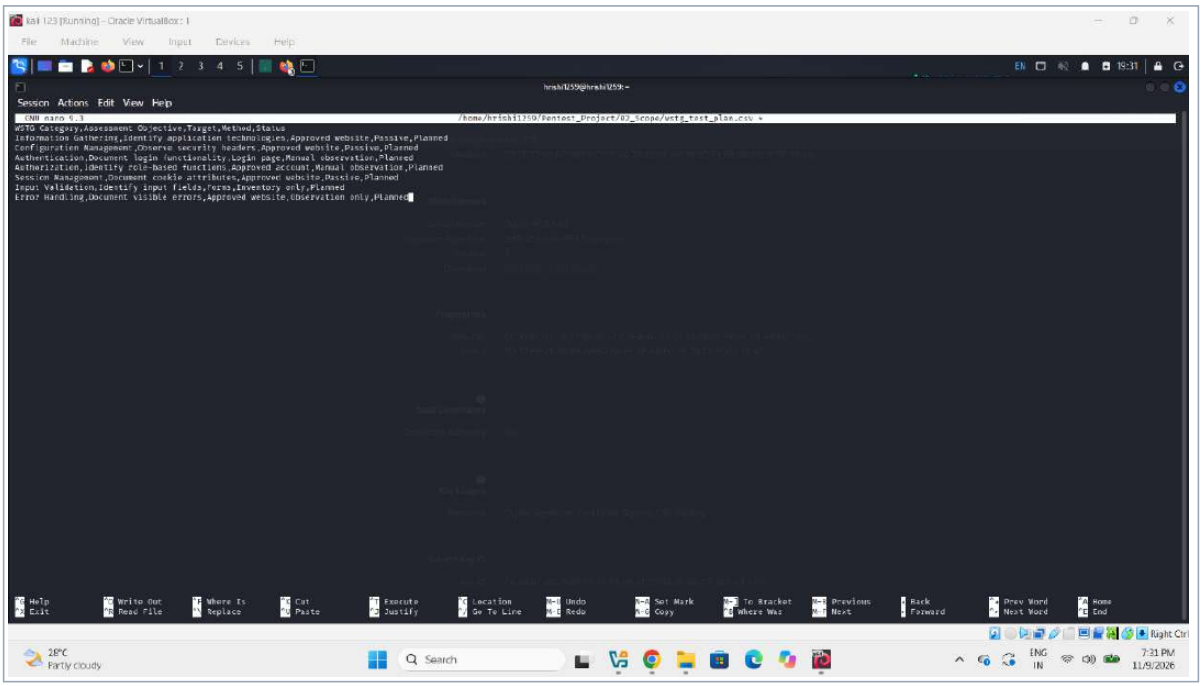


Figure 19: wstg_test_plan.csv opened in nano.

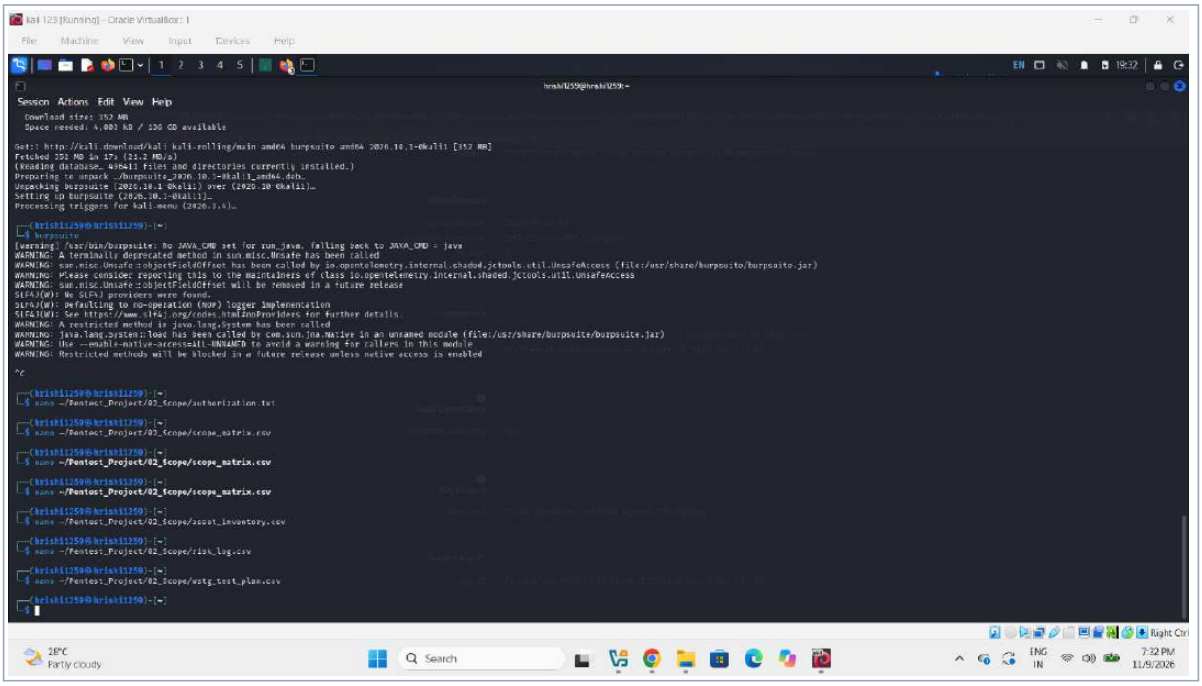


Figure 20: Terminal history showing the Task 2 files created in 02_Scope: authorization.txt, scope_matrix.csv, asset_inventory.csv, risk_log.csv and wstg_test_plan.csv.

Step 14. Verify the URL and redirects

The first technical validation is a single header request to the target URL. The target is stored in a shell variable and the response headers are printed without downloading the page body.

```
TARGET="https://google.com"
curl -sS -D - -o /dev/null "$TARGET"
```

Observation. The request returned **HTTP/2 301** with `location: https://www.google.com/`, which means the address redirects to the www host. Other headers included `server: gws`,

`x-frame-options: SAMEORIGIN`, `x-xss-protection: 0` and a `content-security-policy-report-only` header.

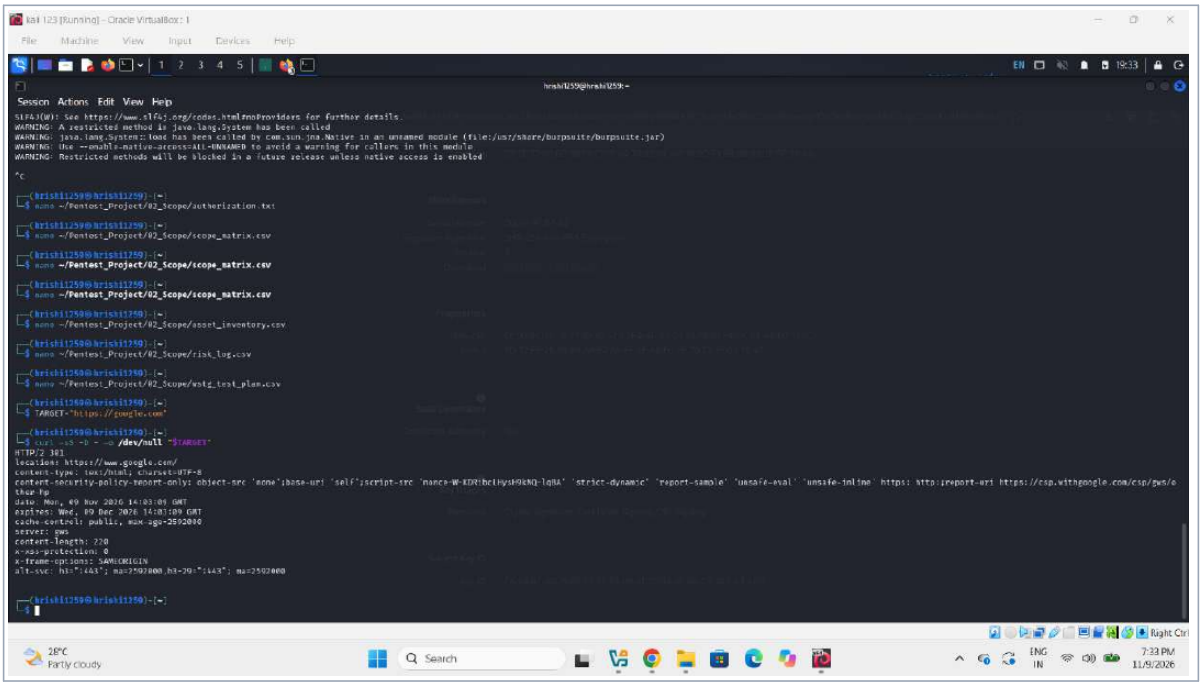


Figure 21: Initial header request: HTTP/2 301 redirect to https://www.google.com/.

The redirect chain was then checked with a maximum of five redirects.

```
curl -sS -L --max-redirs 5 -D - -o /dev/null "$TARGET"
```

The final response was **HTTP/2 200** with `content-type: text/html; charset=ISO-8859-1` and `cache-control: private`. The response set cookies (AEC, NID and a __Secure-STRP cookie) whose attributes include Secure, HttpOnly and SameSite flags. Cookie values are intentionally not reproduced in this report.

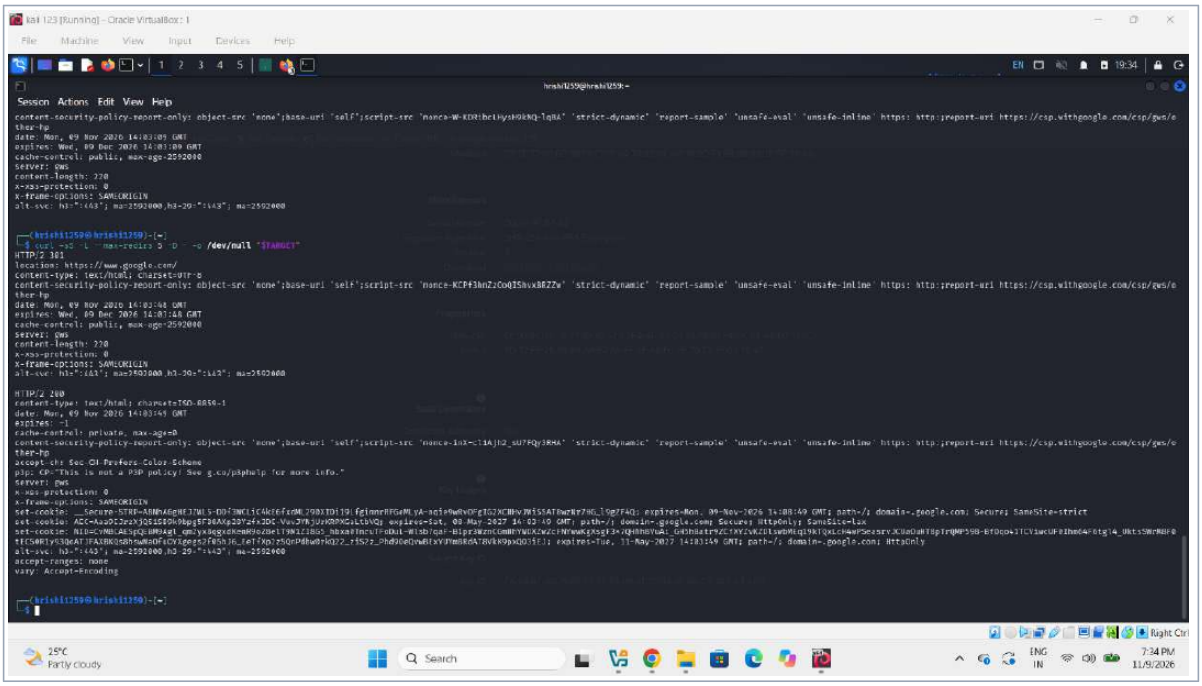


Figure 22: Redirect chain: 301 followed by 200 OK, including Set-Cookie headers (values redacted in this report).

Process note: The guide advises not to follow a redirect until its destination is confirmed to be in scope. Here, the redirect was followed on a public practice site. On a real engagement, confirm the destination host with the coordinator first, then record the results in 02_Scope/url_validation.txt.

6. Task 3 – Reconnaissance and Attack Surface Mapping

Task 3 begins after scope validation. The objective is to understand what technologies the application uses, which pages and functions exist, and how the browser communicates with the server.

Step 15. Start Burp Suite and Firefox

Burp Suite was started with the `burpsuite` command and Firefox was used through the configured proxy. The following were verified: the Burp proxy listener is running, the Firefox proxy is configured, the Burp CA certificate is trusted and HTTP history captures traffic.

Step 16. Identify website technologies

Three methods were used.

Method 1 – Wappalyzer

Wappalyzer was run in Firefox. On the Wappalyzer website it detected technologies such as Google Analytics (GA4), Next.js, Vue.js, Stripe, Google Tag Manager, Crisp Live Chat, UptimeRobot, Nuxt.js and core-js. A CSV export of the detected technologies was also saved and opened in a text editor.

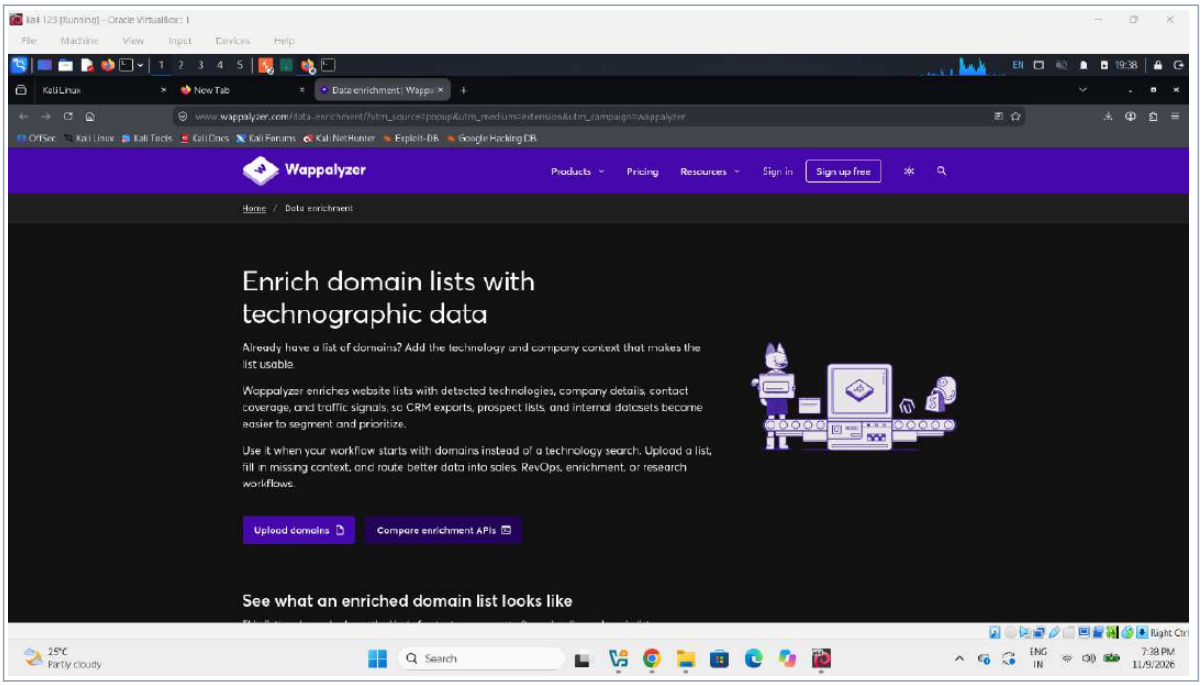


Figure 23: Wappalyzer website opened from the extension (data enrichment page).

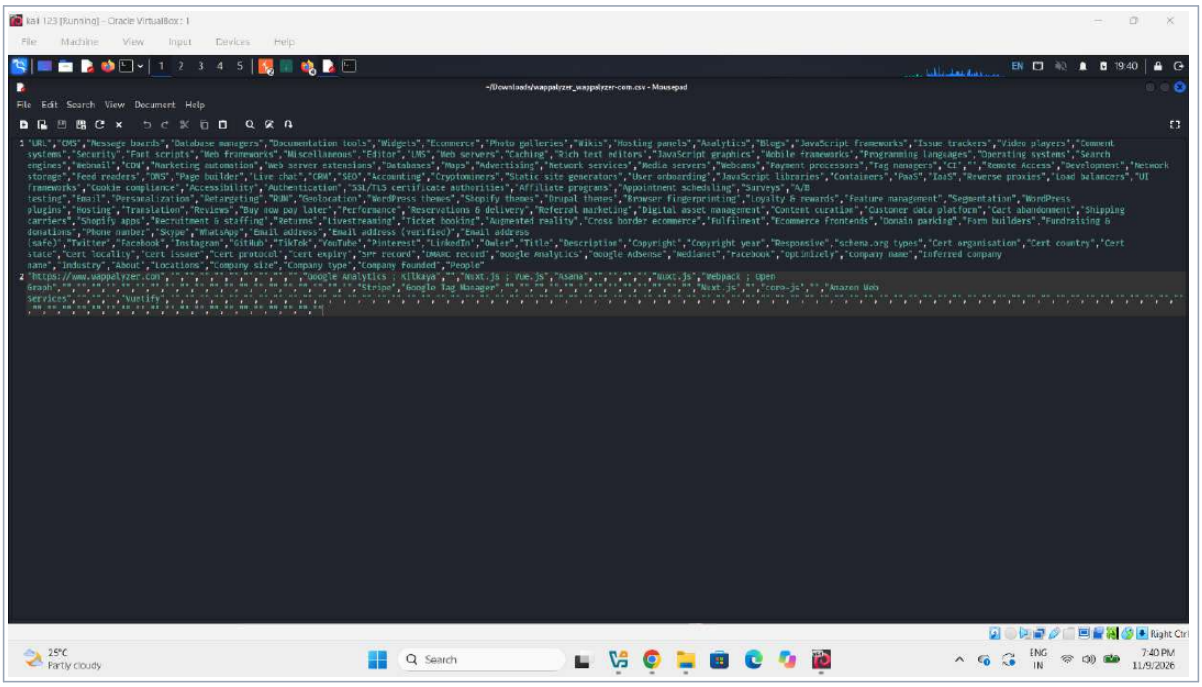


Figure 24: Wappalyzer CSV export (wappalyzer_wappalyzer-com.csv) opened in Mousepad.

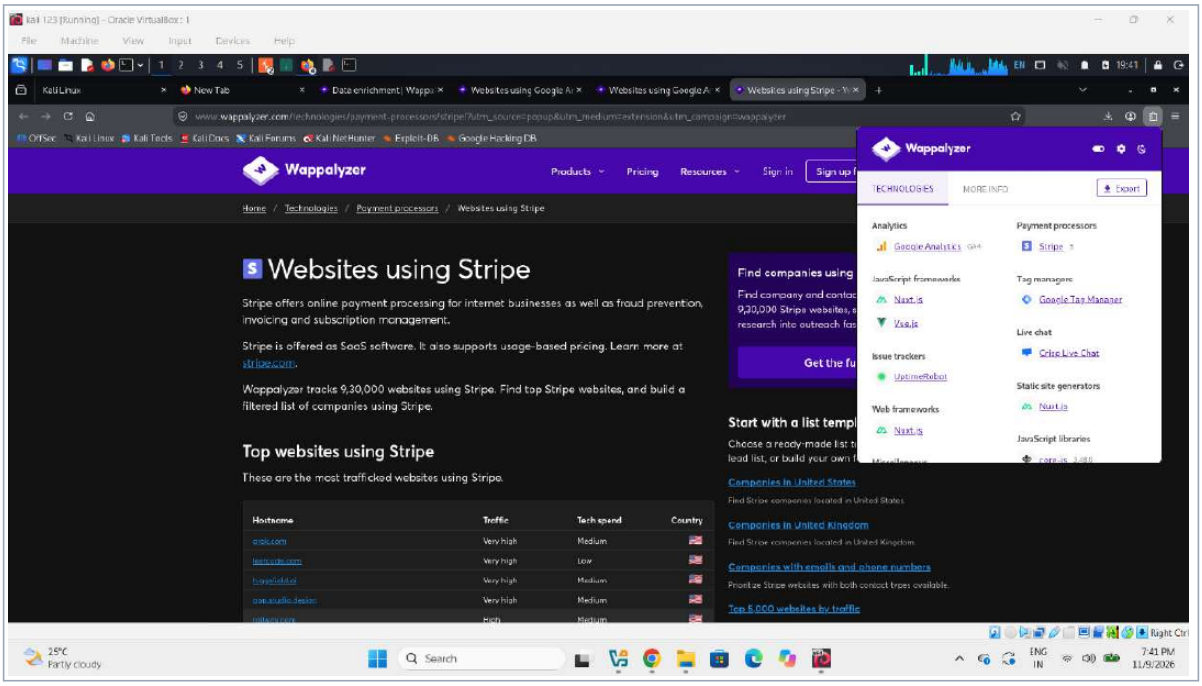


Figure 25: Wappalyzer extension panel listing the technologies detected on the current page.

Method 2 – WhatWeb

```
whatweb --aggression 1 https://google.com
```

WhatWeb (aggression level 1, the least intrusive) reported the following.

URL	Status	Findings
https://google.com	301 Moved Permanently	Country: United States; HTTPServer: gws; RedirectLocation: https://www.google.com/; UncommonHeaders: content-security-policy-report-only, alt-svc; X-Frame-Options: SAMEORIGIN; X-XSS-Protection: 0

URL	Status	Findings
https://www.google.com/	200 OK	Cookies: AEC, NID, __Secure-STRP; HTML5; HTTPServer: gws; HttpOnly: AEC, NID; Script; Title: Google; X-Frame-Options: SAMEORIGIN; X-XSS-Protection: 0

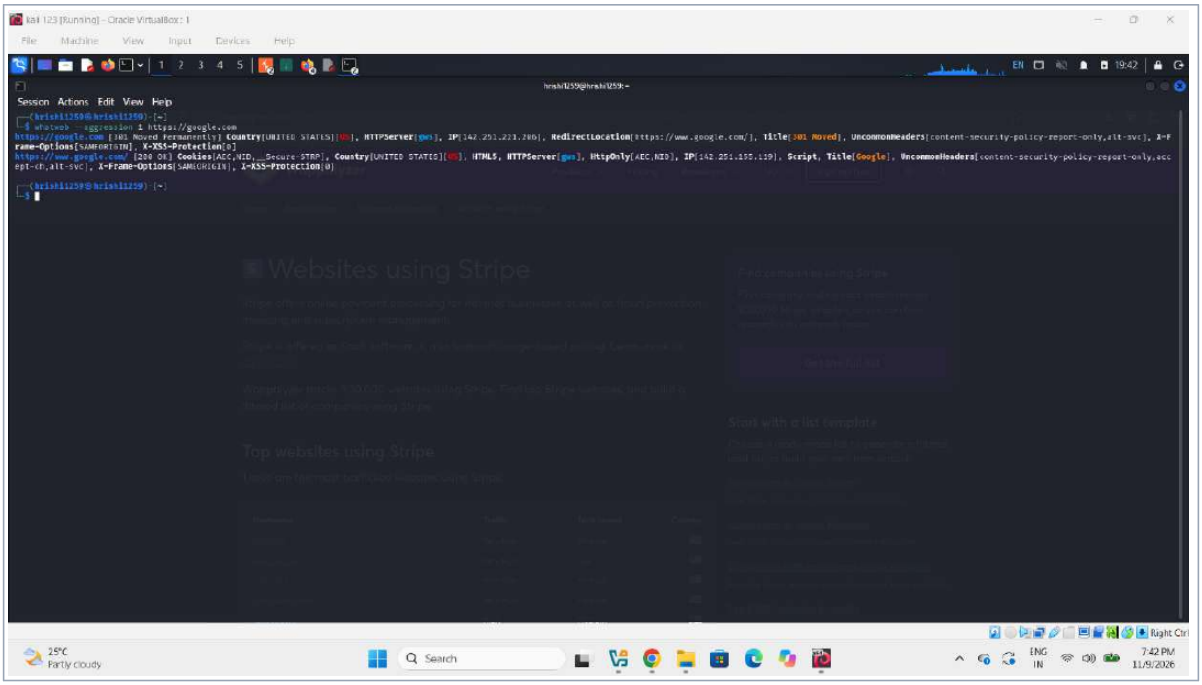


Figure 26: WhatWeb fingerprinting output.

Method 3 – HTTP response headers

```
curl -sSI https://google.com
```

The header request again returned a 301 redirect with `server: gws`, `x-frame-options: SAMEORIGIN`, `x-xss-protection: 0` and a report-only Content-Security-Policy. The absence of some headers, such as X-Powered-By, is an observation and is not by itself proof of a vulnerability.

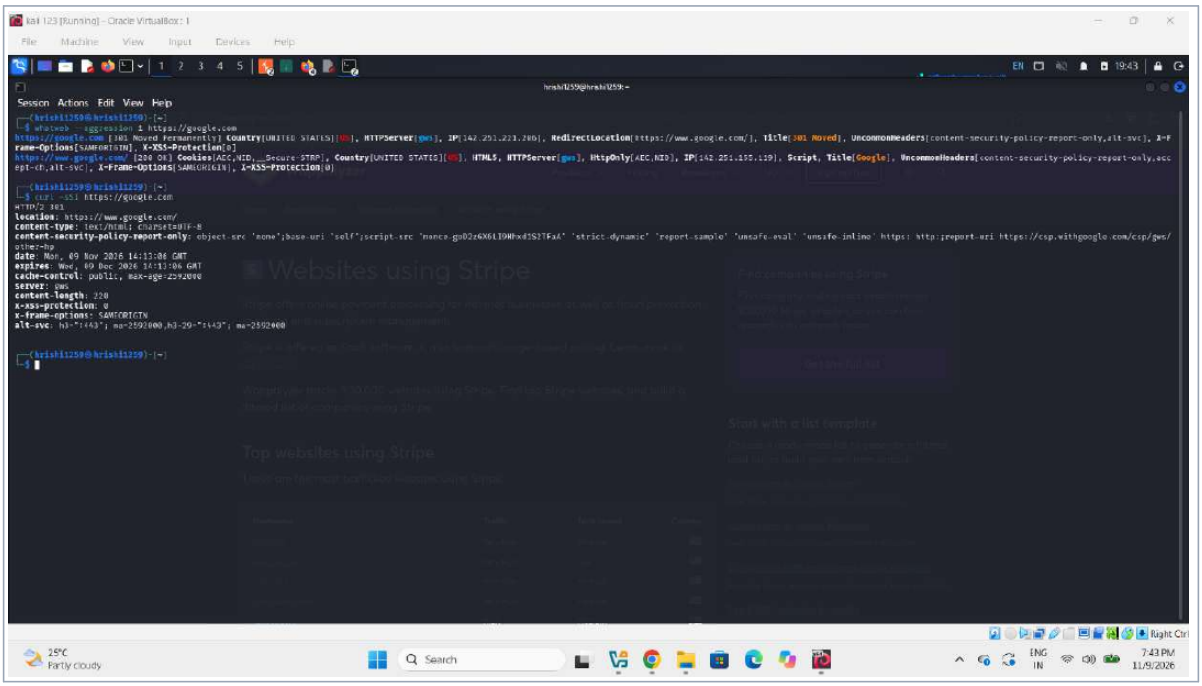


Figure 27: curl -sSI response headers.

Header	Observed value
Server	gws
Content-Type	text/html; charset=UTF-8
X-Frame-Options	SAMEORIGIN
X-XSS-Protection	0
Content-Security-Policy	Report-only policy present (enforced policy not observed)
Strict-Transport-Security	Not observed in this response
X-Powered-By	Not observed

The findings were recorded in `03_Reconnaissance/technology_fingerprint.txt`, which uses the following template: Target, Web Server, Programming Language, CMS, Frontend Framework, JavaScript Libraries, HTTPS, Security Headers, Wappalyzer Findings, WhatWeb Findings and Other Observations.

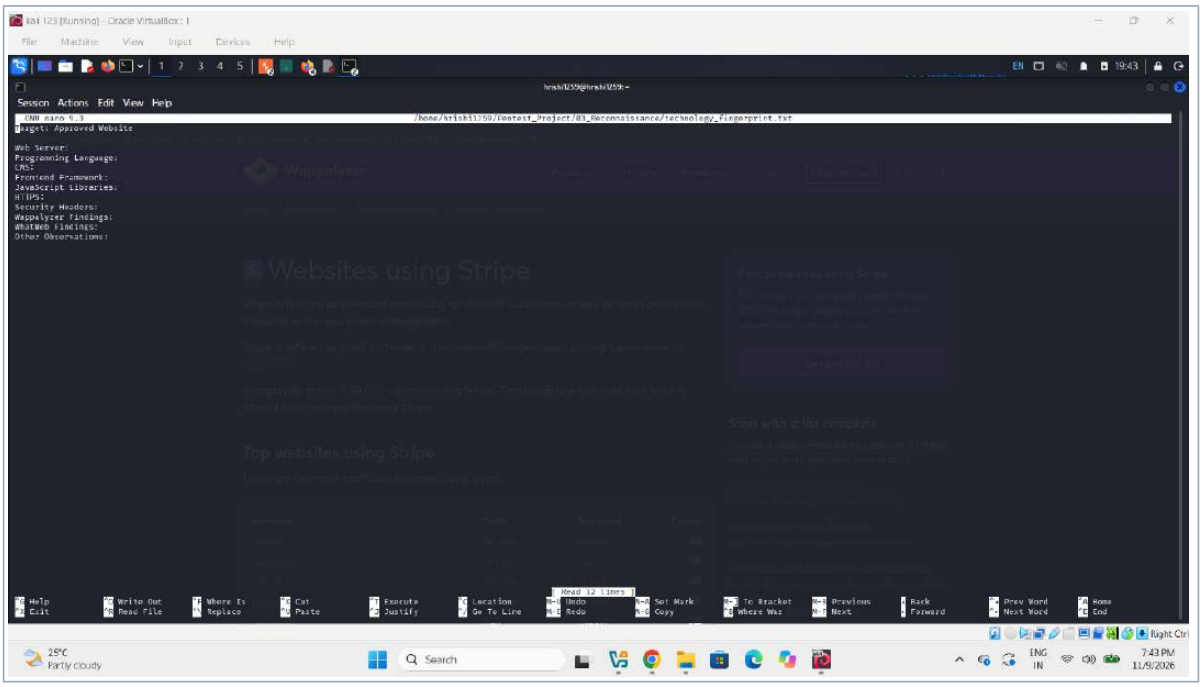


Figure 28: technology_fingerprint.txt template opened in nano.

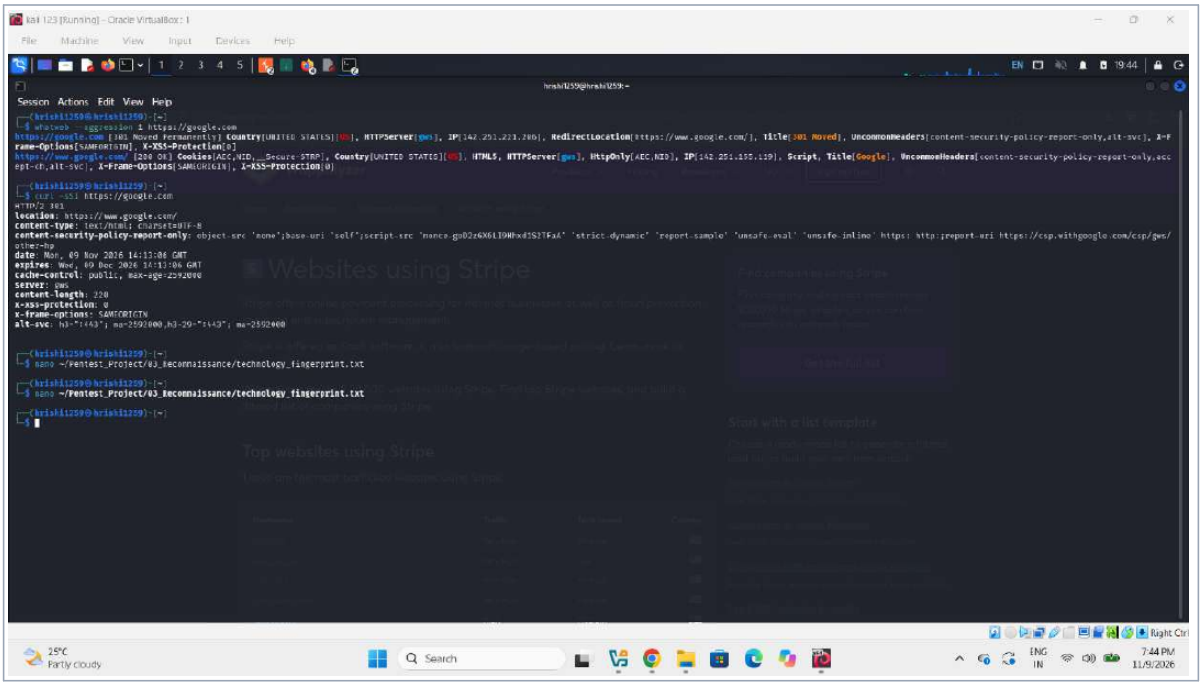


Figure 29: Fingerprinting commands (WhatWeb and curl) with the technology fingerprint file opened for editing.

Step 17. Identify accessible pages

Pages must be identified through normal, authorized navigation only. Hidden URLs must not be guessed and directory discovery must not be attempted. The planned page inventory (`03_Reconnaissance/page_inventory.csv`) is shown below; it is to be completed for the real assigned application.

Page	URL	Functionality
Home	/	Main landing page

Page	URL	Functionality
Login	/login	User authentication
Register	/register	Account registration
Search	/search	Search functionality
Contact	/contact	Contact form
Dashboard	/dashboard	User dashboard, if authorized

Step 18. Capture HTTP requests and responses in Burp Suite

With Intercept switched off, the browser was used normally and the requests were reviewed in **Proxy > HTTP history**. The history shows the HTTP method, host, URL, status code, response length, MIME type and whether TLS was used. Requests to HTTPS hosts appear with the TLS column ticked, which confirms that the Burp CA is working.

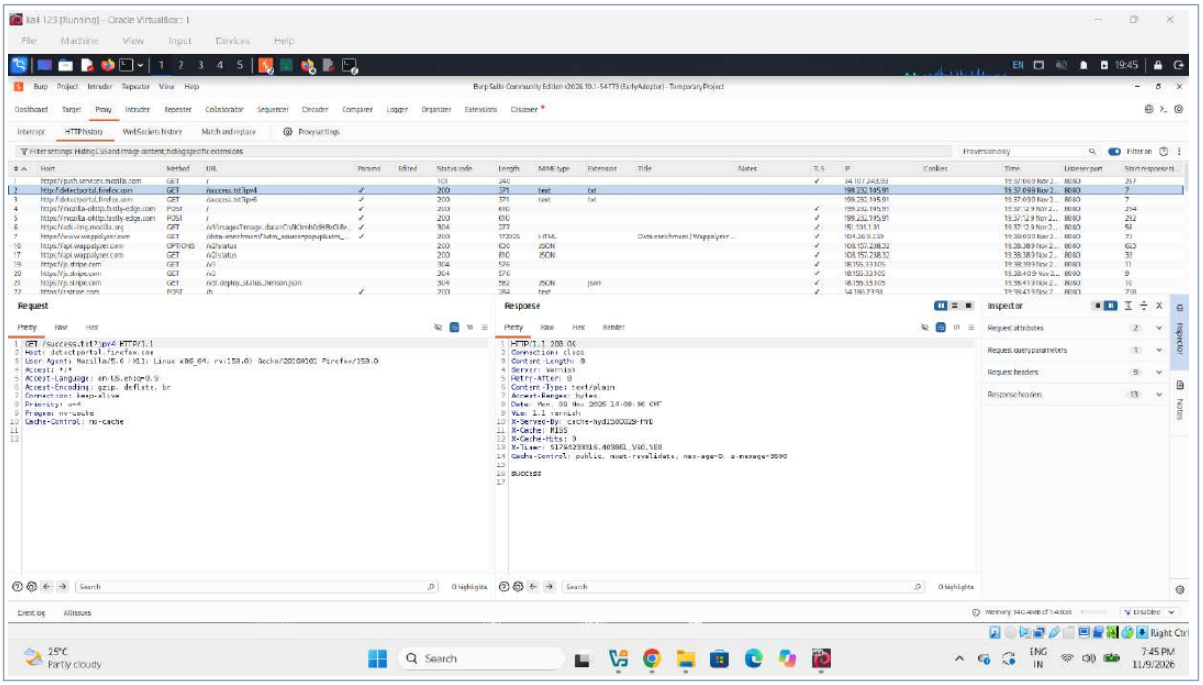


Figure 30: Burp Suite HTTP history. The selected request is a GET to detectportal.firefox.com/success.txt returning 200 OK.

The following table summarizes representative requests visible in the history. Most are background requests generated by Firefox and by the Wappalyzer extension rather than by a target application.

No.	Host	Method	URL	Status	Type
1	push.services.mozilla.com	GET	/	101	–
2	detectportal.firefox.com	GET	/success.txt?ipv4	200	text
3	detectportal.firefox.com	GET	/success.txt?ipv6	200	text
4	mozilla-ohttp.fastly-edge.com	POST	/	200	–
6	ads-img.mozilla.org	GET	/v1/images?image_data=...	304	–
7	www.wappalyzer.com	GET	/data-enrichment/	200	HTML

No.	Host	Method	URL	Status	Type
16	api.wappalyzer.com	OPTIONS	/v2/status	200	JSON
17	api.wappalyzer.com	GET	/v2/status	200	JSON
19	js.stripe.com	GET	/v3	304	–
21	js.stripe.com	GET	/v3/.deploy_status_henson.json	304	JSON

A single request and its response were examined in detail. The request used HTTP/1.1 with a Firefox/153.0 user agent. The response was **HTTP/1.1 200 OK** with `Content-Type: text/plain`, `Server: Varnish`, `X-Cache: MISS` and a body of `success`.

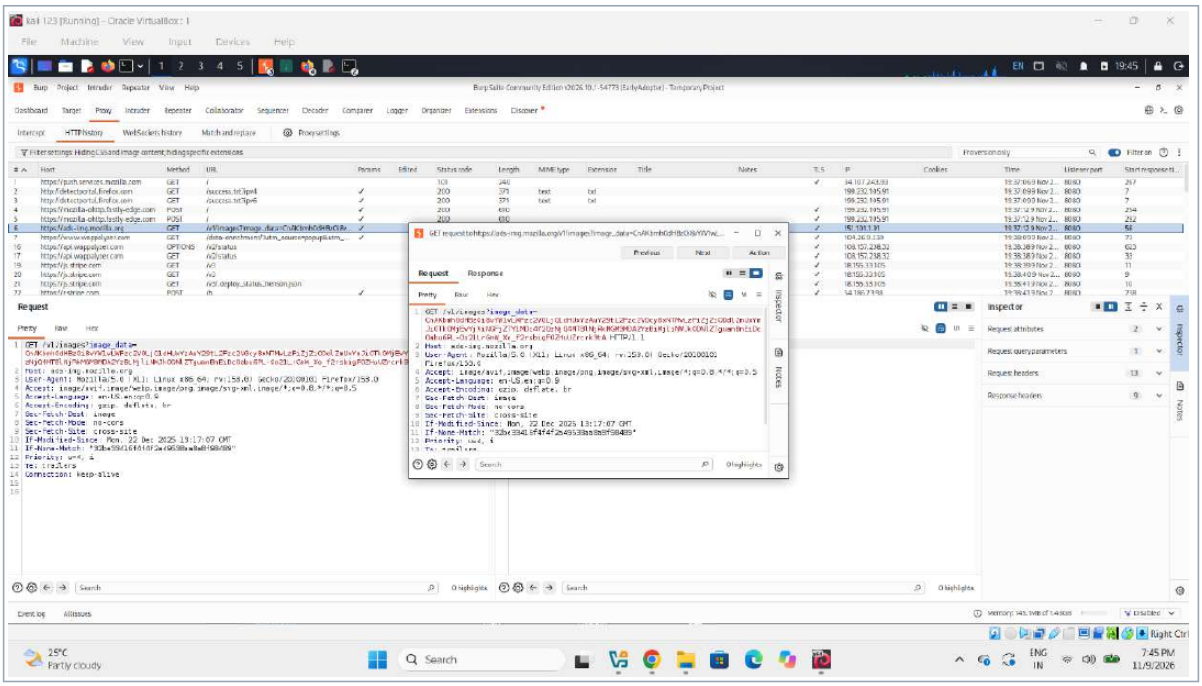


Figure 31: Burp request viewer for GET /v1/images on ads-img.mozilla.org, showing request headers.

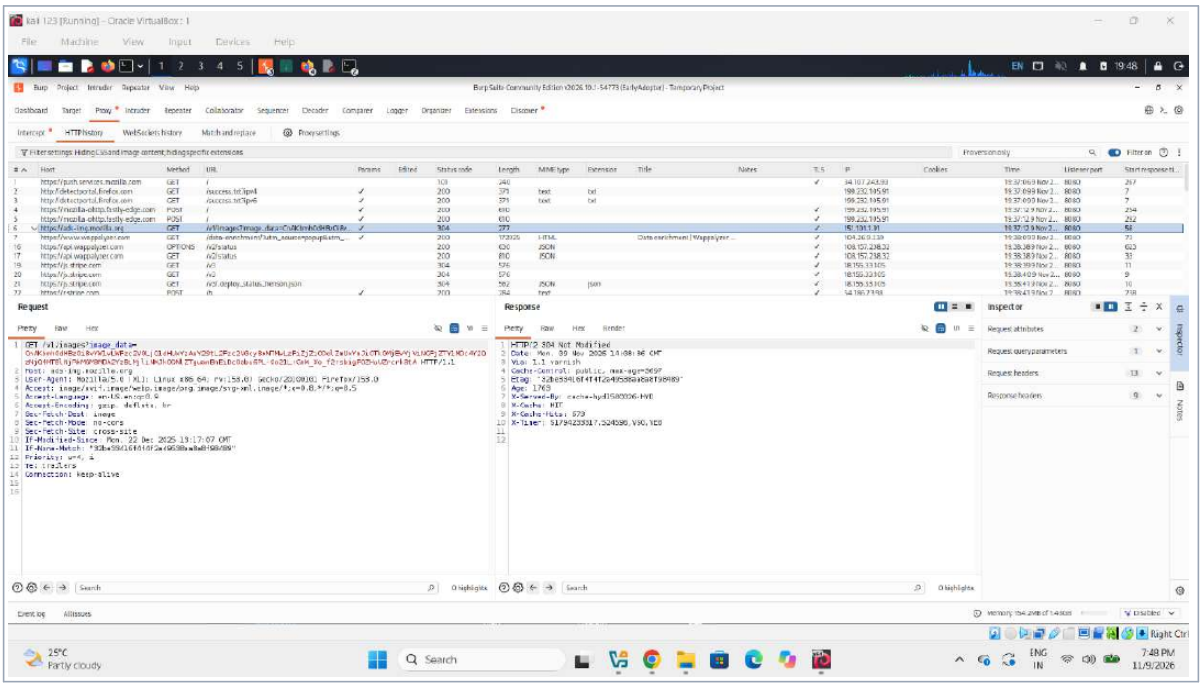


Figure 32: The same request returning 304 Not Modified, with X-Cache: HIT and an ETag header.

Step 19. HTTP methods and status codes

Method	Purpose
GET	Retrieve information
POST	Submit information
PUT	Update or create a resource
DELETE	Delete a resource
PATCH	Partially update a resource
HEAD	Retrieve response headers

Status	Meaning
200	OK
201	Created
301	Permanent redirect
302	Temporary redirect
304	Not modified
400	Bad request
401	Unauthorized
403	Forbidden
404	Not found
405	Method not allowed
500	Internal server error

Status codes observed in this project: **101** (protocol switch), **200** (OK), **301** (redirect) and **304** (not modified).

Step 20. Cookies and security headers

Cookies were reviewed from the response headers. Session tokens must never be copied into the report. The table explains the attributes that should be documented.

Attribute	Purpose
Secure	Sends the cookie over HTTPS only
HttpOnly	Restricts JavaScript access
SameSite	Controls cross-site cookie behavior
Path	Defines the cookie path
Domain	Defines the cookie domain
Expires	Defines the expiration time

Observed (values redacted). The cookies seen on the practice site were set with the domain `.google.com` and path `/`. The __Secure-STRP cookie used Secure and SameSite=strict, while the AEC and NID cookies used HttpOnly; AEC also used Secure and SameSite=lax.

Step 21. Forms and interactive functionality

Features should be explored manually and each HTTP method confirmed in Burp HTTP history rather than assumed.

Function	HTTP method	Authentication
Homepage	GET	No
Login page	GET	No
Login submission	POST	No, before login
User dashboard	GET	Yes
Search	GET or POST	Depends on application
Contact form	POST	Depends on application

Step 22. Endpoint inventory

The endpoint inventory is a mandatory deliverable. The file `03_Reconnaissance/endpoint_inventory.csv` was created with the following illustrative entries, which must be replaced with real observations from the assigned application.

Endpoint	Method	Functionality	Authentication	Status	Notes
/	GET	Homepage	No	200	Public
/login	GET	Login page	No	200	Authentication entry
/login	POST	Login submission	No	302	Redact credentials
/register	GET	Registration page	No	200	Public

Endpoint	Method	Functionality	Authentication	Status	Notes
/search	GET	Search functionality	Depends	200	Query parameters
/dashboard	GET	User dashboard	Yes	200	Authorized account only

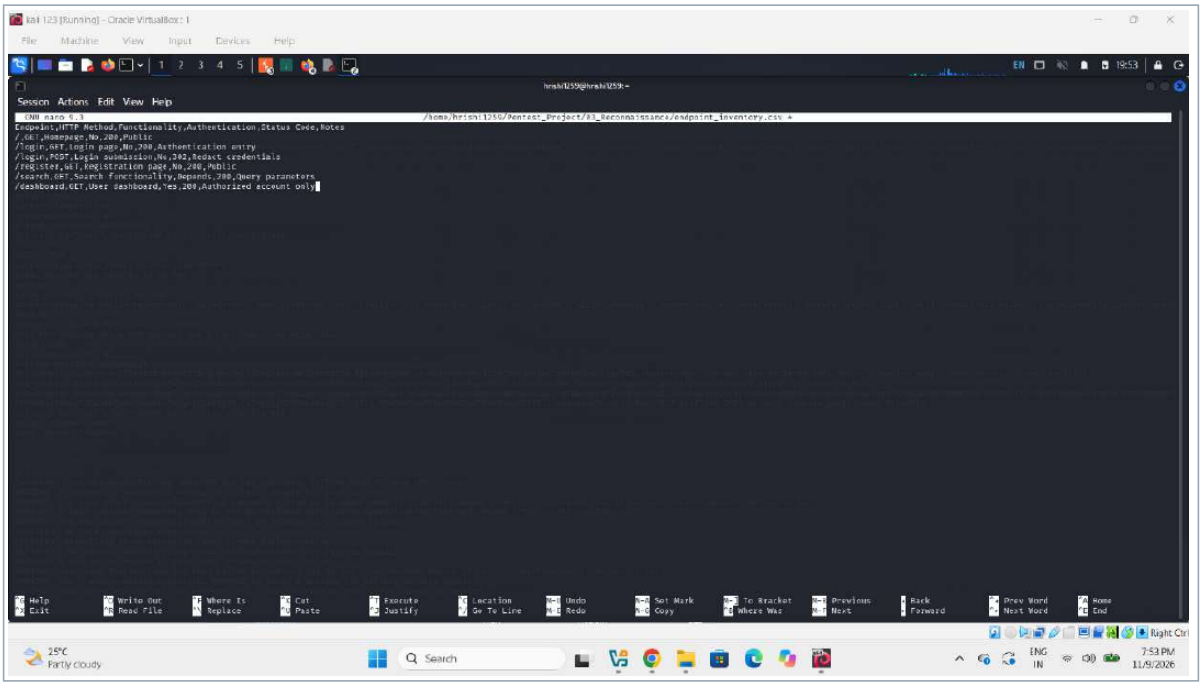


Figure 33: endpoint_inventory.csv opened in nano.

Step 23. HTTP request and response samples

The project requires at least ten sanitized samples, where available. Each sample records the URL, method, request and response headers, status, content type, cookies (redacted) and a brief observation. Eleven requests are visible in the captured history (see the summary table in Step 18). Because the browsing sessions were short and mostly generated background traffic, the samples from the real assigned application remain to be collected. Evidence belongs in `04_Burp_Evidence/http_samples/`, and screenshots in `05_Screenshots/`.

> **Evidence safety:** Never include passwords, access tokens, personal data, API keys or unredacted session cookies in submitted evidence.

Step 24. Passive reconnaissance using Shodan

Shodan indexes information about internet-facing systems that was collected earlier. Searches use previously collected data and do not scan the target. The Shodan home page was opened, and a search page was viewed in the browser.

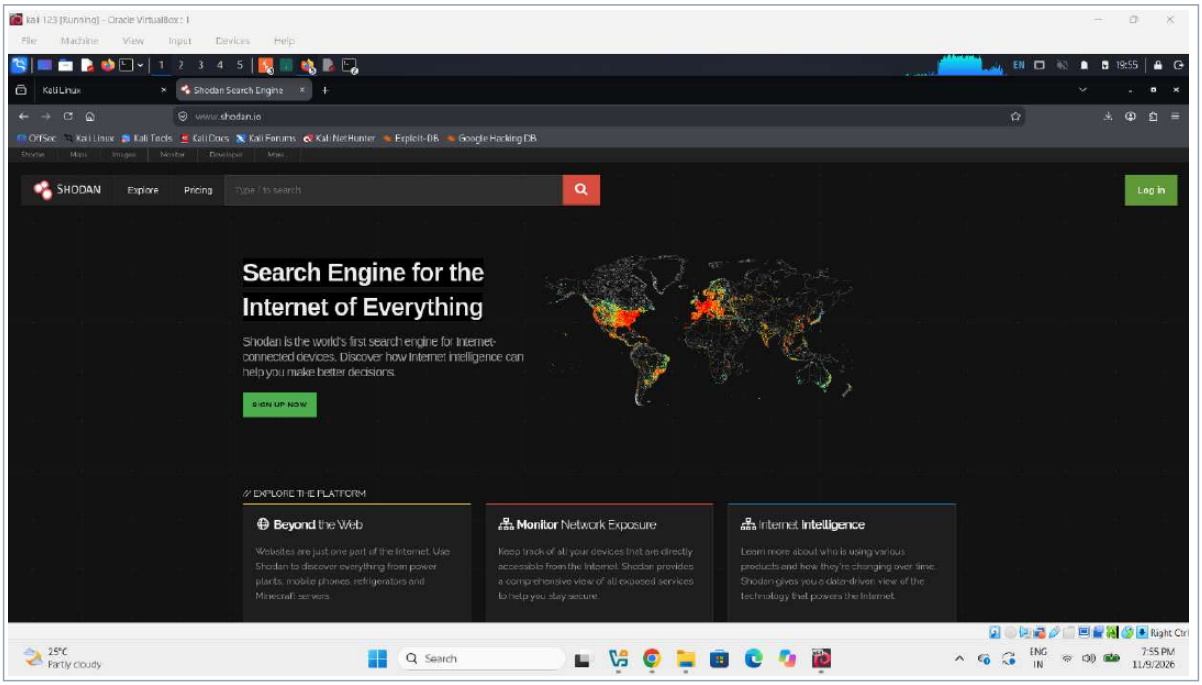


Figure 34: Shodan home page.

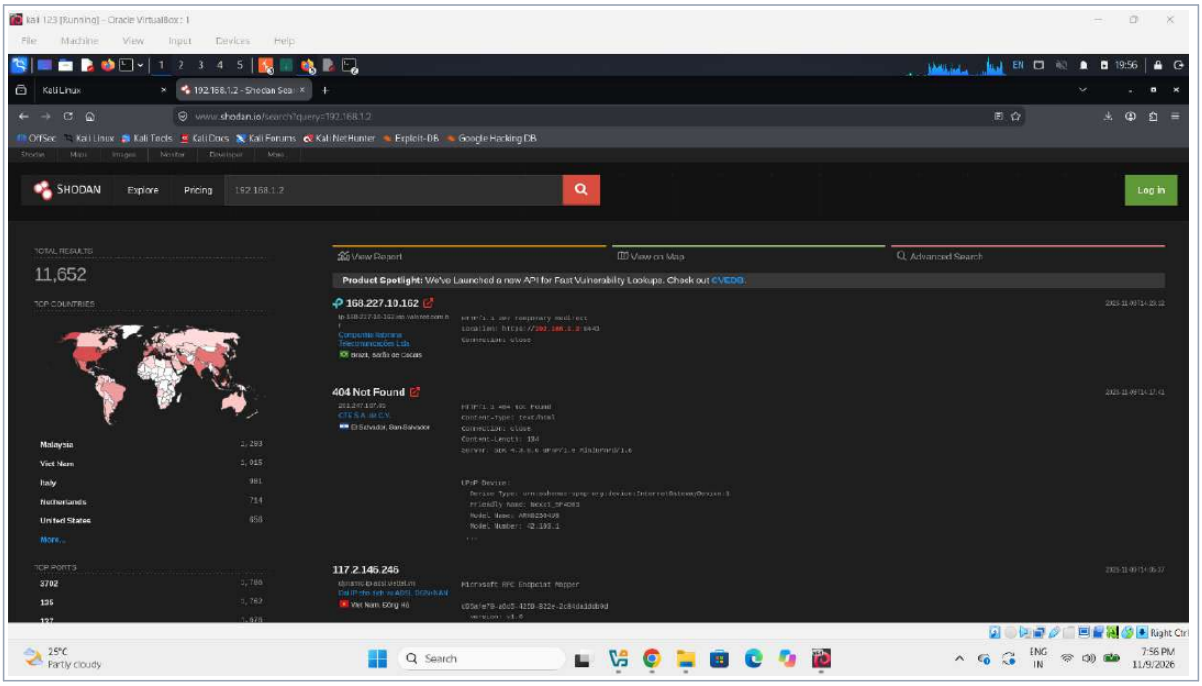


Figure 35: Shodan search results page for the query 192.168.1.2.

> **Interpretation:** 192.168.1.2 is a private (RFC 1918) address. The 11,652 results shown are unrelated public hosts that happen to mention that string in their banners, and they are **not** related to any project asset. Do not treat these results as findings, and do not investigate unrelated third-party systems.

For an approved domain, the optional command line workflow is `pipx install shodan`, `shodan init YOUR_API_KEY` and `shodan domain example.com`. A Shodan-discovered subdomain must not be assumed to be in scope. Observations are saved in `03_Reconnaissance/shodan_findings.txt`.

Step 25. Passive analysis using OWASP ZAP

ZAP was started with the `zaproxy` command. Its main proxy listens on `localhost:8081`, which avoids a conflict with Burp Suite on port 8080. Only one proxy should be used by the browser at a time. The **Manual Explore** function launched a Firefox browser that proxies through ZAP for the URL www.google.com.

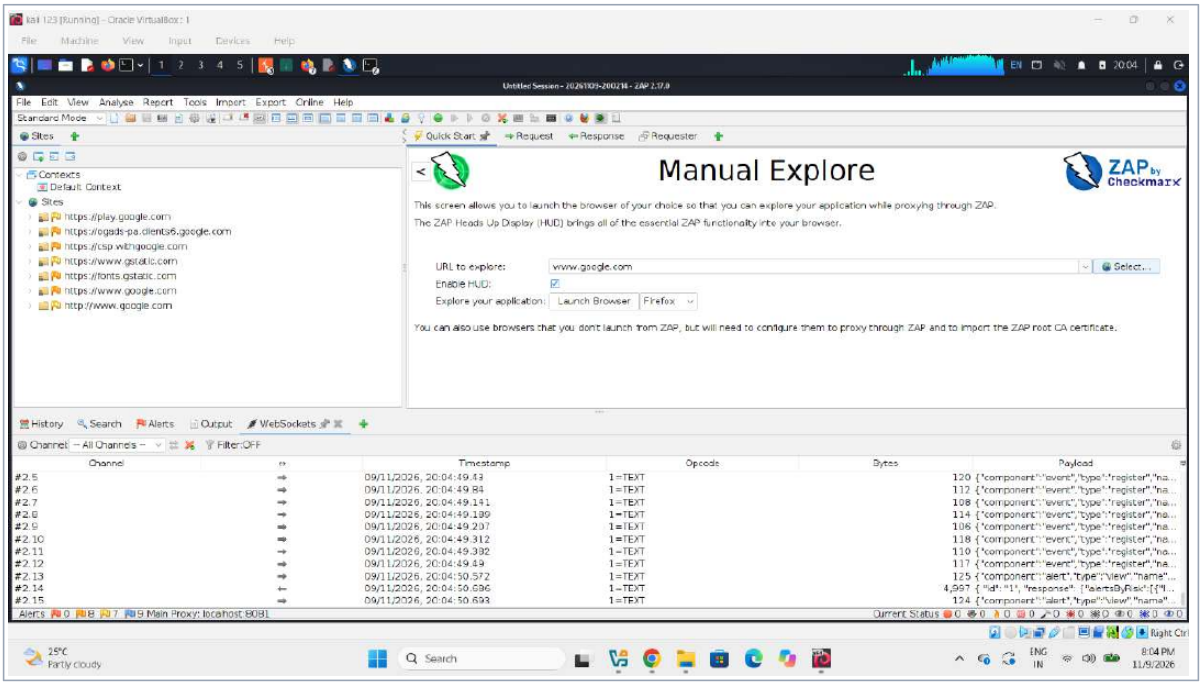


Figure 36: ZAP Manual Explore: URL www.google.com, HUD enabled, and the sites tree populated.

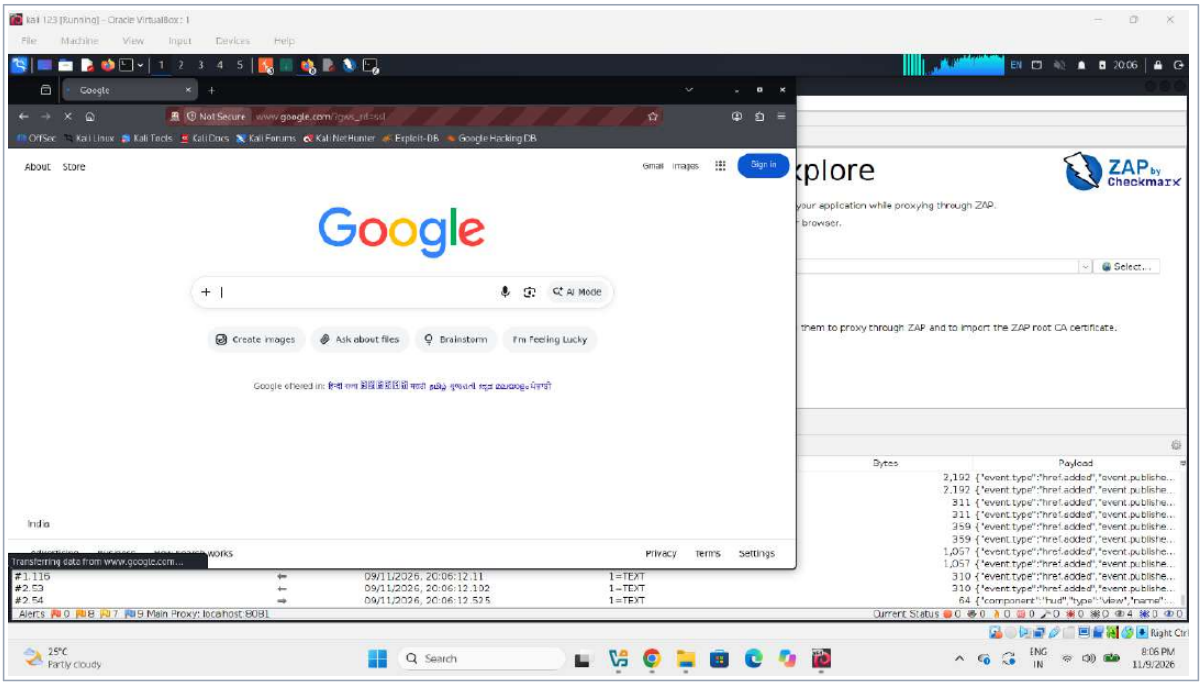


Figure 37: Browser launched from ZAP loading www.google.com through the proxy.

The ZAP Sites tree listed the following hosts contacted while browsing:

Host	Relationship
www.google.com (HTTP and HTTPS)	Primary host
play.google.com	Google-owned subdomain
ogads-pa.clients6.google.com	Google API host
csp.withgoogle.com	CSP reporting
www.gstatic.com	Static content (third party)
fonts.gstatic.com	Fonts (third party)

Passive alert summary. ZAP reported 24 alerts: 0 high, 8 medium, 7 low and 9 informational. Alert groups visible in the Alerts panel were:

- CSP: Failure to define directive with no fallback
- CSP: Wildcard directive; script-src unsafe-eval; script-src unsafe-inline; style-src unsafe-inline
- Content Security Policy (CSP) header not set
- Cross-Domain Misconfiguration (Medium) – passive rule 10098, CWE-264, WASC-14
- Sub Resource Integrity attribute missing
- CSP: Notices
- Cookie without HttpOnly flag
- Cookie with SameSite attribute set to None

The selected example was a Medium-risk **Cross-Domain Misconfiguration** on a font file served from fonts.gstatic.com. The evidence was the header `Access-Control-Allow-Origin: *`. For a public font file this is expected behavior, so this alert is informational in context.

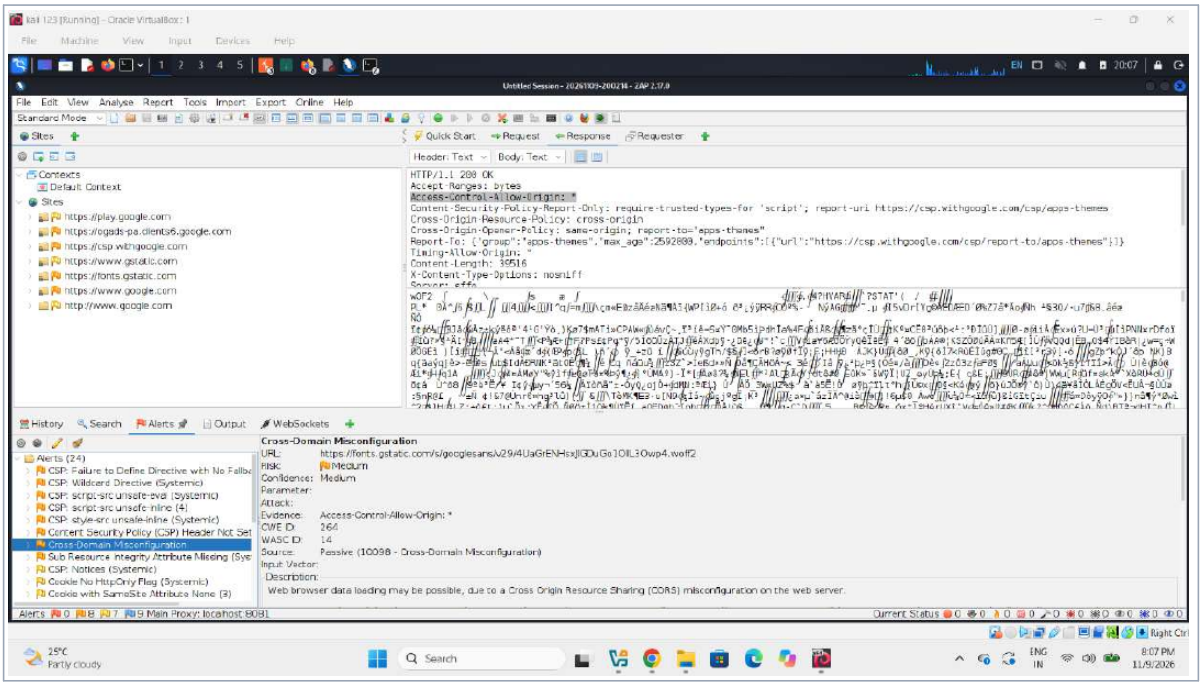
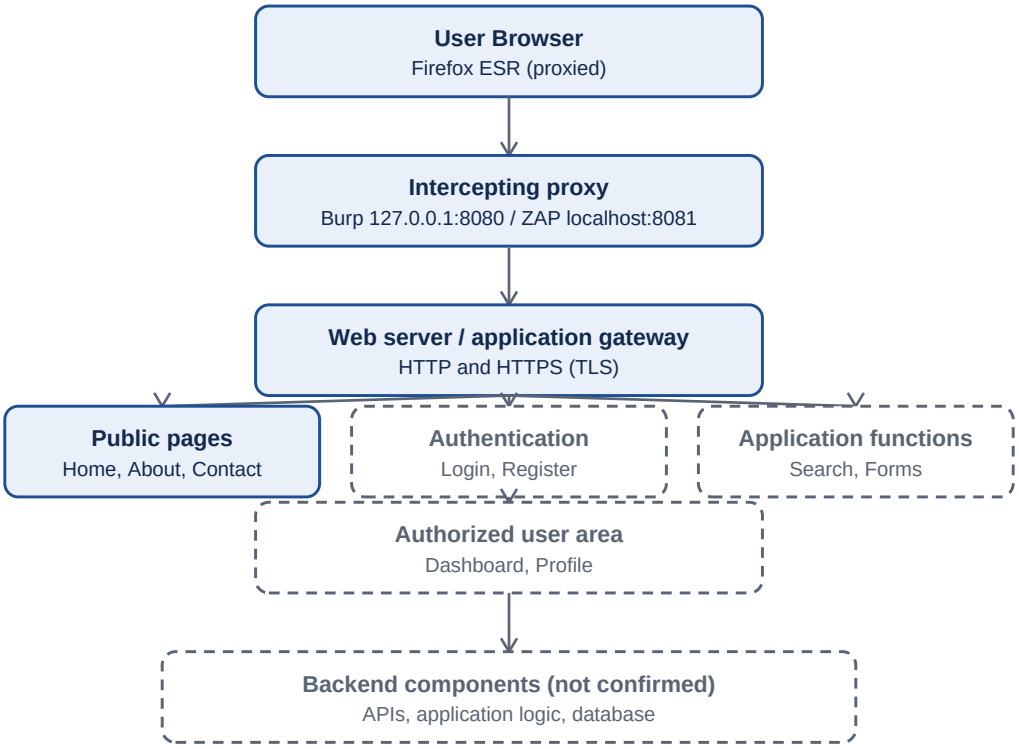


Figure 38: ZAP Alerts panel: 24 passive alerts, with the Cross-Domain Misconfiguration alert selected.

Interpretation: Passive alerts are leads for review, not confirmed vulnerabilities. Many apply to third-party static content that is outside the testing scope, which is why third-party assets are excluded in the risk log (R03). Do not click Attack or Active Scan.

7. Attack Surface Map

The diagram below shows the generic architecture used to plan the assessment. Boxes with a solid outline represent components that were **observed** during this exercise. Boxes with a dashed outline are expected components that have **not been confirmed** and must not be reported as fact without evidence or authorized documentation.



Illustrative architecture only. Replace components with those actually observed or confirmed on the assigned application.

Observed external dependencies (practice site)

Category	Observation
Web server	gws (HTTP response header)
Protocol	HTTP/2 over TLS; alt-svc advertises HTTP/3
Redirect	google.com redirects to https://www.google.com/ (301)
Cookies	AEC, NID, __Secure-STRP
Third-party hosts	www.gstatic.com, fonts.gstatic.com, csp.withgoogle.com
Browser-side security	Report-only CSP, X-Frame-Options SAMEORIGIN

The diagram is saved in the project as `06_Reports/attack_surface_map.md` (Mermaid format) for use in the working files.

8. Summary of Observations

No.	Observation	Source	Type
1	HTTPS inspection works: Burp captures TLS traffic after the PortSwigger CA was trusted	Burp HTTP history	Configuration
2	Target host redirects with HTTP 301 to the www host	curl, WhatWeb	Behavior
3	Server header reports gws; X-Frame-Options is SAMEORIGIN	curl, WhatWeb	Header
4	Content-Security-Policy is delivered in report-only mode	curl	Header
5	Cookies include Secure, HttpOnly and SameSite attributes (values redacted)	curl	Cookie
6	ZAP raised 24 passive alerts (0 high, 8 medium, 7 low, 9 informational)	OWASP ZAP	Passive scan
7	A permissive CORS header (Access-Control-Allow-Origin: *) was seen on a public font file	OWASP ZAP	Passive scan
8	Most captured Burp traffic is Firefox and Wappalyzer background traffic, not target traffic	Burp HTTP history	Data quality
9	Shodan results for a private IP address are unrelated public hosts	Shodan	Data quality

None of the observations above has been validated as a vulnerability. They are documented as reconnaissance data only.

9. Limitations and Compliance Notes

- **Authorization is not yet documented.** Every owner approval in the scope matrix and asset inventory is Pending, and the asset URLs still use the placeholder domain example.com.
- **Practice targets were used.** The URL validation, WhatWeb, curl and ZAP steps used the public site google.com, and the Wappalyzer demonstration used wappalyzer.com. These are third-party sites and are not assigned applications. Only a few low-impact requests and normal browsing were made; no active scanning or intrusive actions took place.
- **Redirect scope was not confirmed** before the redirect was followed.
- **Templates are illustrative.** The page inventory, endpoint inventory, risk log and technology fingerprint files contain template or example values that must be replaced with real observations.
- **Dedicated Firefox profile.** The screenshots do not show a separate testing profile; Firefox reported that the browser is managed by an organization.
- **Sample count.** The ten-sample requirement is met only by background traffic. Samples from the assigned application are still needed.
- **Wappalyzer data sharing** and the CA trust option for email users should be reviewed before client work.

> **Rule:** Do not move from reconnaissance into vulnerability testing or exploitation without written authorization.

10. Recommendations and Next Steps

Priority	Action
1	Obtain written authorization and rules of engagement from the project coordinator, and update 02_Scope/authorization.txt.

Priority	Action
2	Replace example.com placeholders in the asset inventory with the real approved assets and update every Pending status.
3	Repeat URL validation (curl) on the assigned URL and confirm any redirect destination before following it. Save the result in 02_Scope/url_validation.txt.
4	Create a dedicated Firefox testing profile; import the Burp CA only there and only for identifying websites.
5	Browse the assigned application manually and complete page_inventory.csv and endpoint_inventory.csv from real observations.
6	Collect at least ten sanitized HTTP request and response samples from the assigned application and store them in 04_Burp_Evidence/http_samples/.
7	Complete technology_fingerprint.txt, shodan_findings.txt and zap_passive_findings.txt using the assigned domain.
8	Review whether the Wappalyzer data-sharing option is acceptable for the engagement.
9	When the project ends, remove the Burp CA certificate and reset the Firefox proxy settings.

Purpose of each tool

Tool	What it shows
Burp Suite	How the application communicates over HTTP and HTTPS
Firefox	How users interact with the application
curl	HTTP responses inspected from the terminal
Wappalyzer	Technologies the application appears to use
WhatWeb	Additional technology fingerprinting
Shodan	Previously indexed internet-facing information
OWASP ZAP	Passive security observations
OWASP WSTG	Structured methodology for planning security tests

11. Final Submission Checklist

Status key: **Complete** = evidenced in this report; **Partial** = done with limitations; **Pending** = still to do.

Task	Item	Status
Task 1	Linux updated and verified	Complete
Task 1	Firefox testing profile configured	Partial (proxy set; dedicated profile not evidenced)
Task 1	Burp Suite Community installed	Complete
Task 1	OWASP ZAP installed	Complete
Task 1	Burp CA certificate configured	Complete
Task 1	Project folder created	Complete
Task 1	HTTPS inspection verified	Complete
Task 2	Written authorization confirmed	Pending
Task 2	Approved URL and redirects verified	Partial (practice site)

Task	Item	Status
Task 2	Scope matrix completed	Partial (approvals pending)
Task 2	Asset inventory completed	Partial (placeholder URLs)
Task 2	Dependency and risk log completed	Complete
Task 2	OWASP WSTG test plan prepared	Complete
Task 3	Technology fingerprinting documented	Complete
Task 3	Accessible pages identified	Pending
Task 3	Burp HTTP traffic captured	Complete
Task 3	HTTP methods and status codes documented	Complete
Task 3	Cookies and headers documented	Complete
Task 3	Endpoint inventory completed	Partial (template values)
Task 3	At least 10 HTTP samples collected	Partial (background traffic only)
Task 3	Attack surface map prepared	Complete
Task 3	Screenshots and observations saved	Complete
Task 3	ZAP passive observations documented	Complete

Summary: 15 complete, 6 partial and 2 pending out of 23 items.

Appendix A – Tool and System Versions

Component	Version / detail
Operating system	Kali GNU/Linux Rolling, VERSION_ID 2026.3 (Debian-based)
Virtualization	Oracle VirtualBox (VM name: kali 123)
Firefox	Mozilla Firefox 153.4.0esr
curl	8.22.0 (x86_64-pc-linux-gnu), OpenSSL 3.6.5
WhatWeb	0.6.4
Python	3.14.8
Burp Suite	Community Edition 2026.10.1 (package 2026.10.1-0kali1)
OWASP ZAP	2.17.0 (package 2.17.0-0kali1), Java 21 runtime
Wappalyzer	Firefox extension (Technology profiler)
Burp proxy listener	127.0.0.1:8080
ZAP main proxy	localhost:8081
Burp CA certificate	PortSwigger CA, valid 8 Nov 2014 to 8 Nov 2036

Appendix B – Project Folder Structure

```
Pentest_Project/
|-- 01_Setup/
|   |-- project_notes.txt
|   `-- tool_versions.txt
|-- 02_Scope/
|   |-- authorization.txt
|   |-- scope_matrix.csv
|   |-- asset_inventory.csv
|   |-- risk_log.csv
|   |-- url_validation.txt
|   `-- wstg_test_plan.csv
|-- 03_Reconnaissance/
|   |-- technology_fingerprint.txt
|   |-- page_inventory.csv
|   |-- endpoint_inventory.csv
|   |-- shodan_findings.txt
|   `-- zap_passive_findings.txt
|-- 04_Burp_Evidence/
|   `-- http_samples/
|-- 05_Screenshots/
|   |-- burp_proxy.png
|   |-- firefox_proxy.png
|   |-- https_certificate.png
|   |-- wappalyzer.png
|   `-- http_history.png
|-- 06_Reports/
|   |-- attack_surface_map.md
|   `-- reconnaissance_report.pdf
`-- 07_Logs/
```

End of report.